

C++ Standard Library Issues to be moved in San Diego

Doc. no. P1224R0

Date: Revised 2018-10-08 at 04:58:18 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[2183^{\(0\)}](#). Muddled allocator requirements for `match_results` constructors

Section: 31.10.1 [\[re.results.const\]](#), 31.10.6 [\[re.results.all\]](#) **Status:** [Ready](#) **Submitter:** Pete Becker **Opened:** 2012-08-29 **Last modified:** 2018-06-11

Priority: 3

View other [active issues](#) in [\[re.results.const\]](#).

View all other [issues](#) in [\[re.results.const\]](#).

View all issues with [Ready](#) status.

Discussion:

31.10.1 [\[re.results.const\]](#) p1 says:

In all `match_results` constructors, a copy of the `Allocator` argument shall be used for any memory allocation performed by the constructor or member functions during the lifetime of the object.

There are three constructors:

```
match_results(const Allocator& = Allocator());  
match_results(const match_results& m);  
match_results(match_results&& m) noexcept;
```

The second and third constructors do not have an `Allocator` argument, so despite the "all `match_results` constructors", it is not possible to use "the `Allocator` argument" for the second and third constructors.

The requirements for those two constructors also does not give any guidance. The second constructor has no language about allocators, and the third states that the stored `Allocator` value is move constructed from `m.get_allocator()`, but doesn't require using that allocator to allocate memory.

The same basic problem recurs in 31.10.6 [\[re.results.all\]](#), which gives the required return value for `get_allocator()`:

Returns: A copy of the `Allocator` that was passed to the object's constructor or, if that allocator has been replaced, a copy of the most recent replacement.

Again, the second and third constructors do not take an `Allocator`, so there is nothing that meets this requirement when those constructors are used.

[2018-06-02, Daniel comments and provides wording]

The introductory wording of `match_results` says in 31.10 [\[re.results\]](#) p2:

The class template `match_results` satisfies the requirements of an allocator-aware container and of a sequence container (26.2.1, 26.2.3) except that only operations defined for const-qualified sequence containers are supported and that the semantics of comparison functions are different from those required for a container.

This wording essentially brings us to 26.2.1 [\[container.requirements.general\]](#) which describes in p8 in general the usage of allocators:

[...] Copy constructors for these container types obtain an allocator by calling `allocator_traits<allocator_type>::select_on_container_copy_construction` on the allocator belonging to the container being copied. Move constructors obtain an allocator by move construction from the allocator belonging to the container being moved. [...]

The constructors referred to in the issue discussion are the copy constructor and move constructor of `match_results`, so we know already what the required effects are supposed to be.

26.2.1 [\[container.requirements.general\]](#) p8 also says more to this allocator topic a bit latter:

[...] All other constructors for these container types take a `const allocator_type&` argument. [Note: If an invocation of a constructor uses the default value of an optional allocator argument, then the `Allocator` type must support value-initialization. — end note] A copy of this allocator is used for any memory allocation and element construction performed, by these constructors and by all member functions, during the lifetime of each container object or until the allocator is replaced. [...]

Further requirements imposed on two of the three `match_results` constructors can be derived from Table 80 — "Allocator-aware container requirements" via the specified expressions

```
X()  
X(m)  
X(rv)
```

In other words: The existing wording does already say *everything* that it said by 31.10.1 [\[re.results.const\]](#) p1 (end even more), except for possibly the tiny problem that

```
match_results(const Allocator& a = Allocator());
```

uses "const Allocator&" instead of "const allocator_type&" in the signature, albeit even that deviation shouldn't change the intended outcome, which is IMO crystal-clear when looking at sub-clauses 26.2.1 [\[container.requirements.general\]](#) and 26.2.3 [\[sequence.reqmts\]](#) as a whole.

That directly makes two mutually exclusive solutions feasible:

- Either strike 31.10.1 [\[re.results.const\]](#) p1 completely; or
- Replace 31.10.1 [\[re.results.const\]](#) p1 by referring to the specification of allocators in 26.2.1 [\[container.requirements.general\]](#) and 26.2.3 [\[sequence.reqmts\]](#).

My suggestion is to favour for the first option, because attempting to provide extra wording that refers to allocators and the three constructors may lead to the false impression that no further allocator-related requirements hold for type `match_results` which are not explicitly repeated here again.

[2018-06, Rapperswil]

The group agrees with the provided resolution. Move to Ready.

Proposed resolution:

This wording is relative to [N4750](#).

1. Edit 31.10.1 [\[re.results.const\]](#) as indicated:

~~1- In all `match_results` constructors, a copy of the `Allocator` argument shall be used for any memory allocation performed by the constructor or member functions during the lifetime of the object.~~

[2184^{\(i\)}](#). Muddled allocator requirements for `match_results` assignments

Section: 31.10.1 [\[re.results.const\]](#), 31.10.6 [\[re.results.all\]](#) **Status:** [Ready](#) **Submitter:** Pete Becker **Opened:** 2012-08-29 **Last modified:** 2018-06-11

Priority: 3

View other [active issues](#) in [\[re.results.const\]](#).

View all other [issues](#) in [\[re.results.const\]](#).

View all issues with [Ready](#) status.

Discussion:

The effects of the two assignment operators are specified in Table 141. Table 141 makes no mention of allocators, so, presumably, they don't touch the target object's allocator. That's okay, but it leaves the question: `match_results::get_allocator()` is supposed to return "A copy of the `Allocator` that was passed to the object's constructor or, if that allocator has been replaced, a copy of the most recent replacement"; if assignment doesn't replace the allocator, how can the allocator be replaced?

[2018-06-04, Daniel comments and provides wording]

Similar to the reasoning provided in the 2018-06-02 comment in LWG [2183](#), it is possible to refer to the introductory wording of `match_results` which says in 31.10 [\[re.results\]](#) p2:

The class template `match_results` satisfies the requirements of an allocator-aware container and of a sequence container (26.2.1, 26.2.3) except that only operations defined for const-qualified sequence containers are supported and that the semantics of comparison functions are different from those required for a container.

Again, similar to LWG [2183](#), this allows us to deduce the required effects of the copy/move assignment operators discussed here, because 26.2.1 [\[container.requirements.general\]](#) p8 also says:

[...] The allocator may be replaced only via assignment or `swap()`. Allocator replacement is performed by copy assignment, move assignment, or swapping of the allocator only if `allocator_traits<allocator_type>::propagate_on_container_copy_assignment::value`, `allocator_traits<allocator_type>::propagate_on_container_move_assignment::value`, or `allocator_traits<allocator_type>::propagate_on_container_swap::value` is true within the implementation of the corresponding container operation. In all container types defined in this Clause, the member `get_allocator()` returns a copy of the allocator used to construct the container or, if that allocator has been replaced, a copy of the most recent replacement. [...]

So this wording already specifies everything we need, *except* for the problem that 31.10 [\[re.results\]](#) p2 quoted above restricts to operations supported by a const-qualified sequence container, which of-course would exclude the copy assignment and the move assignment operators. But given that these mutable definitions are defined for `match_results`, it seems that the only fix needed is to adjust 31.10 [\[re.results\]](#) p2 a bit to ensure that both assignment operators are covered (again) by the general allocator-aware container wording.

[2018-06, Rapperswil]

The group generally likes the suggested direction, but would prefer the changed wording to say effectively "except that only copy assignment, move assignment, and operations defined...". Once applied, move to ready.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4750](#).

1. Edit 31.10 [\[re.results\]](#) as indicated:

-2- The class template `match_results` satisfies the requirements of an allocator-aware container and of a sequence container (26.2.1 [\[container.requirements.general\]](#), 26.2.3 [\[sequence.reqmts\]](#)) except that [besides copy assignment and move assignment](#) only operations

defined for const-qualified sequence containers are supported and that the semantics of comparison functions are different from those required for a container.

[2018-06-06, Daniel updates wording]

Proposed resolution:

This wording is relative to [N4750](#).

1. Edit 31.10 [\[re.results\]](#) as indicated:

-2- The class template `match_results` satisfies the requirements of an allocator-aware container and of a sequence container (26.2.1 [\[container.requirements.general\]](#), 26.2.3 [\[sequence.reqmts\]](#)) except that only [copy assignment, move assignment, and](#) operations defined for const-qualified sequence containers are supported and that the semantics of comparison functions are different from those required for a container.

[2412](#)⁽ⁱ⁾. `promise::set_value()` and `promise::get_future()` should not race

Section: 33.6.6 [\[futures.promise\]](#), 33.6.10.1 [\[futures.task.members\]](#) **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2014-06-23 **Last modified:** 2018-06-11

Priority: 3

View other [active issues](#) in [\[futures.promise\]](#).

View all other [issues](#) in [\[futures.promise\]](#).

View all issues with [Ready](#) status.

Discussion:

The following code has a data race according to the standard:

```
std::promise<void> p;
std::thread t{ []{
    p.get_future().wait();
}};
p.set_value();
t.join();
```

The problem is that both `promise::set_value()` and `promise::get_future()` are non-const member functions which modify the same object, and we only have wording saying that the `set_value()` and `wait()` calls (i.e. calls setting and reading the shared state) are synchronized.

The calls don't actually access the same memory locations, so the standard should allow it. My suggestion is to state that calling `get_future()` does not conflict with calling the various functions that make the shared state ready, but clarify with a note that this does not imply any synchronization or "happens before", only being free from data races.

[2015-02 Cologne]

Handed over to SG1.

[2016-10-21, Nico comments]

After creating a promise or packaged task one thread can call `get_future()` while another thread can set values/exceptions (either directly or via function call). This happens very easily.

Consider:

```
promise<string> p;
thread t(doSomething, ref(p));
cout << "result: " << p.get_future().get() << endl;
```

AFAIK, this is currently UB due to a data race (calling `get_future()` for the promise might happen while setting the value in the promise).

Yes, a fix is pretty easy:

```
promise<string> p;
future<string> f(p.get_future());
thread t(doSomething, ref(p));
cout << "result: " << f.get() << endl;
```

but I would like to have `get_future()` and setters be synchronized to avoid this UB.

This would especially make the use of packaged tasks a lot easier. Consider:

```
vector<packaged_task<int(char)>> tasks;
packaged_task<int(char)> t1(func);

// start separate thread to run all tasks:
auto futCallTasks = async(launch::async, callTasks, ref(tasks));

for (auto& fut : tasksResults) {
    cout << "result: " << fut.get_future().get() << endl; // OOPS: UB
}
```

Again, AFAIK, this program currently is UB due to a data race. Instead, currently I'd have to program, which is a lot less convenient:

```

vector<packaged_task<int(char)>> tasks;
vector<future<int>> tasksResults;
packaged_task<int(char)> t1(func);
tasksResults.push_back(t1.getFuture());
tasks.push_back(move(t1));

// start separate thread to run all tasks:
auto futCallTasks = async(launch::async, callTasks, ref(tasks));

for (auto& fut : tasksResults) {
    cout << "result: " << fut.get() << endl;
}

```

With my naive thinking I see not reason not to guarantee that these calls synchronize (as `get_future` returns an "address/reference" while all setters set the values there).

Previous resolution [SUPERSEDED]:

This wording is relative to N3936.

1. Change 33.6.6 [futures.promise] around p12 as indicated:

```
future<R> get_future();
```

-12- *Returns:* A future<R> object with the same shared state as *this.

-?- *Synchronization:* Calls to this function do not conflict (6.8.2 [intro.multithread]) with calls to `set_value`, `set_exception`, `set_value at thread exit`, or `set_exception at thread exit`. [Note: Such calls need not be synchronized, but implementations must ensure they do not introduce data races. — end note]

-13- *Throws:* future_error if *this has no shared state or if get_future has already been called on a promise with the same shared state as *this.

-14- *Error conditions:* [...]

2. Change 33.6.10.1 [futures.task.members] around p13 as indicated:

```
future<R> get_future();
```

-13- *Returns:* A future<R> object that shares the same shared state as *this.

-?- *Synchronization:* Calls to this function do not conflict (6.8.2 [intro.multithread]) with calls to `operator()` or `make_ready at thread exit`. [Note: Such calls need not be synchronized, but implementations must ensure they do not introduce data races. — end note]

-14- *Throws:* a future_error object if an error occurs.

-15- *Error conditions:* [...]

[2017-02-28, Kona]

SG1 has updated wording for LWG 2412. SG1 voted to move this to Ready status by unanimous consent.

[2017-03-01, Kona, SG1]

GeoffR to forward revised wording.

[2018-06, Rapperswil, Wednesday evening session]

JW: lets move on and I'll file another issue to make the wording better

BO: the current wording is better than what there before

JM: ACTION I should file an editorial issue to clean up on how to refer to [res.on.data.races]: raised editorial issue 2097

ACTION: move to Ready

Daniel rebases wording to N4750.

Proposed resolution:

This wording is relative to [N4750](#).

1. Change 33.6.6 [futures.promise] around p12 as indicated:

```
future<R> get_future();
```

-12- *Returns:* A future<R> object with the same shared state as *this.

-?- *Synchronization:* Calls to this function do not introduce data races (6.8.2 [intro.multithread]) with calls to `set_value`, `set_exception`, `set_value at thread exit`, or `set_exception at thread exit`. [Note: Such calls need not synchronize with each other. — end note]

-13- *Throws:* future_error if *this has no shared state or if get_future has already been called on a promise with the same shared state as *this.

-14- *Error conditions:* [...]

2. Change 33.6.10.1 [futures.task.members] around p13 as indicated:

```
future<R> get_future();
```

-13- *Returns*: A future object that shares the same shared state as `*this`.

-?- *Synchronization*: Calls to this function do not introduce data races (6.8.2 [intro.multithread]) with calls to `operator()` or `make_ready_at_thread_exit`. [Note: Such calls need not synchronize with each other. — end note]

-14- *Throws*: a `future_error` object if an error occurs.

-15- *Error conditions*: [...]

2682⁽ⁱ⁾. `filesystem::copy()` won't create a symlink to a directory

Section: 30.11.14.3 [fs.op.copy] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-04-19 **Last modified:** 2018-06-11

Priority: 2

View other [active issues](#) in [fs.op.copy].

View all other [issues](#) in [fs.op.copy].

View all issues with [Ready](#) status.

Discussion:

(First raised in c++std-lib-38544)

`filesystem::copy` doesn't create a symlink to a directory in this case:

```
copy("/", "root", copy_options::create_symlinks);
```

If the first path is a file then a symlink is created, but I think my implementation is correct to do nothing for a directory. We get to bullet 30.11.14.3 [fs.op.copy] (3.6) where `is_directory(f)` is true, but `options == create_symlinks`, so we go to the next bullet (3.7) which says "Otherwise, no effects."

I think the case above should either create a symlink, or should report an error. GNU `cp -s` gives an error in this case, printing "omitting directory '/". An error seems reasonable, you can use `create_symlink` to create a symlink to a directory.

[2016-05 Issues Telecon]

This is related to [2681](#); and should be considered together.

[2016-08 Chicago]

Wed AM: Move to Tentatively Ready

Previous resolution [SUPERSEDED]:

Add a new bullet following (3.6) in 30.11.14.3 [fs.op.copy] as shown:

- If `!exists(t)`, then `create_directory(to, from)`.
- Then, iterate over the files in `from`, as if by `for (directory_entry& x : directory_iterator(from))`, and for each iteration `copy(x.path(), to/x.path().filename(), options | copy_options::unspecified )`
- Otherwise, if `is_directory(f) && (options & copy_options::create_symlinks) != copy_options::none`, then report an error with an error code argument equal to `make_error_code(errc::is_a_directory)`.
- Otherwise, no effects.

[2016-10-16, Eric reopens and provides improved wording]

The current PR makes using `copy(...)` to copy/create a directory symlink an error. For example, the following is now an error:

```
copy("/", "root", copy_options::create_symlinks);
```

However the current PR doesn't handle the case where both `copy_options::create_symlinks` and `copy_options::recursive` are specified. This case is still incorrectly handled by bullet (3.6) [fs.op.copy].

I suggest we move the PR before this bullet so that it catches the recursive copy case, since currently the conditions are ordered:

```
3.6 Otherwise if is_directory(f) && (bool(options & copy_options::recursive) || ...)  
3.X Otherwise if is_directory(f) && bool(options & copy_options::create_symlinks)
```

So 3.6 catches `create_symlinks | recursive` but I believe we want 3.X to handle it instead.

[2018-01-26 issues processing telecon]

Status to 'Review'; we think this is OK but want some implementation experience before adopting it.

[2018-01-29 Jonathan says]

The proposed resolution for LWG 2682 has been in GCC's Filesystem TS implementation for more than a year. It's also in our `std::filesystem` implementation on the subversion trunk.

[2018-06; Rapperswil Wednesday evening]

JW: can we use the words we are shipping already since two years?

BO: what we got is better than what we had before

no objection to moving to Ready

ACTION move to Ready

ACTION link 2682 and LWG [3057](#) and set a priority 2 and look at 3057 in San Diego

Daniel rebases wording to N4750.

Proposed resolution:

This wording is relative to [N4750](#).

1. Add a new bullet before (4.8) in 30.11.14.3 [\[fs.op.copy\]](#) as shown:

(4.7) — Otherwise, if `is_regular_file(f)`, then:

[...]

(4.2) — Otherwise, if

```
is_directory(f) &&
(options & copy_options::create_symlinks) != copy_options::none
```

```
then report an error with an error_code argument equal to make_error_code(errc::is_a_directory).
```

(4.8) — Otherwise, if

```
is_directory(f) &&
((options & copy_options::recursive) != copy_options::none ||
options == copy_options::none)
```

then:

(4.8.1) — If `exists(t)` is false, then `create_directory(to, from)`.

(4.8.2) — Then, iterate over the files in `from`, as if by

```
for (const directory_entry& x : directory_iterator(from))
  copy(x.path(), to/x.path().filename(),
    options | copy_options::in-recursive-copy);
```

where *in-recursive-copy* is a bitmask element of `copy_options` that is not one of the elements in 30.11.9.3 [\[fs.enum.copy.opts\]](#).

(4.9) — Otherwise, for the signature with argument `ec`, `ec.clear()`.

(4.10) — Otherwise, no effects.

[2697^{\(i\)}](#). [\[concurr.ts\]](#) Behavior of `future/shared_future` unwrapping constructor when given an invalid future

Section: 99 [\[concurr.ts::futures.unique_future\]](#), 99 [\[concurr.ts::futures.shared_future\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2016-04-22 **Last modified:** 2018-06-11

Priority: 2

View other [active issues](#) in [\[concurr.ts::futures.unique_future\]](#).

View all other [issues](#) in [\[concurr.ts::futures.unique_future\]](#).

View all issues with [Ready](#) status.

Discussion:

Addresses: [concurr.ts](#)

In the concurrency TS, the `future/shared_future` unwrapping constructors

```
future(future<future<R>>&&) noexcept;
shared_future(future<shared_future<R>>&& rhs) noexcept;
```

appear to implicitly require `rhs` be valid (e.g., by referring to its shared state, and by requiring a `valid() == true` postcondition). However, they are also marked `noexcept`, suggesting that they are wide-contract, and also makes the usual suggested handling for invalid futures, throwing a `future_error`, impossible.

Either the `noexcept` should be removed, or the behavior with an invalid future should be specified.

Original resolution alternative #1 [NOT CHOSEN]:

This wording is relative to [N4577](#).

Strike the `noexcept` on these constructors in 99 [\[concurr.ts::futures.unique_future\]](#)/1-2 and 99 [\[concurr.ts::futures.shared_future\]](#)/1-2, and optionally add a *Requires*: `rhs.valid() == true` paragraph.

[2016-11-12, Issaquah]

Sat PM: We prefer alternative #2 - Move to review

[2018-06; Rapperswil, Wednesday evening session]

DR: there is a sentence ended followed by an entirely new sentence

JM: so the period should be a semicolon in both edits

MC: ACTION I can make the change editorially

ACTION move to Ready

Proposed resolution:

This wording is relative to [N4577](#).

Alternative #2: Specify that an empty (shared_)future object is constructed if rhs is invalid, and adjust the postcondition accordingly.

1. Edit 99 [concurr.ts::futures.unique_future] as indicated:

```
future(future<future<R>>&& rhs) noexcept;
```

-3- Effects: **If `rhs.valid() == false`, constructs an empty future object that does not refer to a shared state. Otherwise, `c` constructs a future object from the shared state referred to by `rhs`.** ~~F~~; the future becomes ready when one of the following occurs:

- Both the rhs and `rhs.get()` are ready. The value or the exception from `rhs.get()` is stored in the future's shared state.
- rhs is ready but `rhs.get()` is invalid. An exception of type `std::future_error`, with an error condition of `std::future_errc::broken_promise` is stored in the future's shared state.

-4- Postconditions:

- **`valid() == true` `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.**
- `rhs.valid() == false`.

2. Edit 99 [concurr.ts::futures.shared_future] as indicated:

```
shared_future(future<shared_future<R>>&& rhs) noexcept;
```

-3- Effects: **If `rhs.valid() == false`, constructs an empty shared future object that does not refer to a shared state. Otherwise, `c` constructs a shared_future object from the shared state referred to by `rhs`.** ~~F~~; the shared_future becomes ready when one of the following occurs:

- Both the rhs and `rhs.get()` are ready. The value or the exception from `rhs.get()` is stored in the shared_future's shared state.
- rhs is ready but `rhs.get()` is invalid. The shared_future stores an exception of type `std::future_error`, with an error condition of `std::future_errc::broken_promise`.

-4- Postconditions:

- **`valid() == true` `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.**
- `rhs.valid() == false`.

[2936^{\(i\)}](#). Path comparison is defined in terms of the generic format

Section: 30.11.7.4.8 [[fs.path.compare](#)] **Status:** [Ready](#) **Submitter:** Billy Robert O'Neal III **Opened:** 2017-02-21 **Last modified:** 2018-06-11

Priority: 2

View all issues with [Ready](#) status.

Discussion:

Currently, path comparison is defined elementwise, which implies a conversion from the native format (implied by `native()` returning `const string&`). However, the conversion from the native format to the generic format might not be information preserving. This would allow two paths `a` and `b` to say `a.compare(b) == 0`, but `a.native().compare(b.native()) != 0` as a result of this missing information, which is undesirable. We only want that condition to happen if there are redundant directory separators. We also don't want to change the path comparison to be in terms of the native format, due to Peter Dimov's example where we want `path("a/b")` to sort earlier than `path("a.b")`, and we want `path("a/b") == path("a/////b")`.

Citing a Windows example, conversion to the generic format is going to have to drop alternate data streams. This might give `path("a/b:ads") == path("a/b")`. I think I should consider the alternate data streams as part of the path element though, so this case might be fine, so long as I make `path("b:ads").native()` be `"b:ads"`. This might not work for our z/OS friends though, or for folks where the native format looks nothing like the generic format.

Additionally, this treats root-directory specially. For example, the current spec wants `path("c:/a/b") == path("c:/a/////b")`, but `path("c:/a/b") != path("c:///a/b")`, because `native()` for the root-directory path element will literally be the slashes or preferred separators.

This addresses similar issues to those raised in US 57 — it won't make absolute paths sort at the beginning or end but it will make paths of the same kind sort together.

[2017-03-04, Kona Saturday morning]

We decided that this had seen so much churn that we would postpone looking at this until Toronto

[2017-07 Toronto Thurs Issue Prioritization]

Priority 2

[2016-07, Toronto Saturday afternoon issues processing]

Billy to reword after Davis researches history about ordering. Status to Open.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4640](#).

1. Make the following edits to 30.11.7.4.8 [fs.path.compare]:

```
int compare(const path& p) const noexcept;
```

-1- Returns:

— Let `rootNameComparison` be the result of `this->root_name().native().compare(p.root_name().native())`. If `rootNameComparison` is not 0, `rootNameComparison`; otherwise,

— If `this->has_root_directory()` and `!p.has_root_directory()`, a value less than 0; otherwise,

— If `!this->has_root_directory()` and `p.has_root_directory()`, a value greater than 0; otherwise,

— A value greater than, less than, or equal to 0, ordering the paths in a depth-first traversal order.

-?- [Note: For POSIX and Windows platforms, this is accomplished by lexicographically ordering the half-open ranges `[begin(), end())` of `this->relative_path()` and `p.relative_path()` as follows:

— A value less than 0, if `native()` for the elements of `*this->relative_path()` are lexicographically less than `native()` for the elements of `p.relative_path()`; otherwise,

— a value greater than 0, if `native()` for the elements of `*this->relative_path()` are lexicographically greater than `native()` for the elements of `p.relative_path()`; otherwise,

— 0.

— end note

~~-2- Remarks: The elements are determined as if by iteration over the half-open range `[begin(), end())` for `*this` and `p`.~~

```
int compare(const string_type& s) const
int compare(basic_string_view<value_type> s) const;
```

~~-3- Returns: `compare(path(s))`~~

[Editor's note: Delete paragraph 3 entirely and merge the `value_type` overload with those above.]

```
int compare(const value_type* s) const
```

-4- ~~Returns~~ Effects: Equivalent to return `compare(path(s))`;.

[2018-01-26 issues processing telecon]

Status set to 'Review'. We like the wording, but would like to see some implementation experience.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4659](#).

1. Make the following edits to 30.11.7.4.8 [fs.path.compare]:

```
int compare(const path& p) const noexcept;
```

-1- Returns:

— Let `rootNameComparison` be the result of `this->root_name().native().compare(p.root_name().native())`. If `rootNameComparison` is not 0, `rootNameComparison`; otherwise,

— If `this->has_root_directory()` and `!p.has_root_directory()`, a value less than 0; otherwise,

— If `!this->has_root_directory()` and `p.has_root_directory()`, a value greater than 0; otherwise,

— ~~a value less than 0~~, if `native()` for the elements of `*this->relative_path()` are lexicographically less than `native()` for the elements of `p.relative_path()`, ~~a value less than 0~~; otherwise,

— ~~a value greater than 0~~, if `native()` for the elements of `*this->relative_path()` are lexicographically greater than `native()` for the elements of `p.relative_path()`, ~~a value greater than 0~~; otherwise,

```

— 0.

2- Remarks: The elements are determined as if by iteration over the half-open range [begin(), end()) for *this and p.

int compare(const string_type& s) const
int compare(basic_string_view<value_type> s) const;

3- Returns: compare(path(s))

[Editor's note: Delete paragraph 3 entirely and merge the value_type overload with those above.]

int compare(const value_type* s) const

-4- ReturnsEffects: Equivalent to: return compare(path(s));.

```

[2018-02-13 Billy improves wording]

The revised wording has the effect to invert the ordering of the added new bullets (2) and (3), the effect of this change is that

```
path("c:/").compare("c:")
```

compares greater, not less.

[2018-06, Rapperswil Wednesday evening]

Agreement to move that to Ready, Daniel rebased to N4750.

Proposed resolution:

This wording is relative to [N4750](#).

1. Make the following edits to 30.11.7.4.8 [\[fs.path.compare\]](#):

```
int compare(const path& p) const noexcept;
```

-1- Returns:

— Let `rootNameComparison` be the result of `this->root_name().native().compare(p.root_name().native())`. If `rootNameComparison` is not 0, `rootNameComparison`; otherwise,

— If `!this->has_root_directory()` and `p.has_root_directory()`, a value less than 0; otherwise,

— If `this->has_root_directory()` and `!p.has_root_directory()`, a value greater than 0; otherwise,

— a value less than 0, if `native()` for the elements of `*this->relative_path()` are lexicographically less than `native()` for the elements of `p.relative_path()`, a value less than 0; otherwise,

— a value greater than 0, if `native()` for the elements of `*this->relative_path()` are lexicographically greater than `native()` for the elements of `p.relative_path()`, a value greater than 0; otherwise,

— 0.

~~2- Remarks: The elements are determined as if by iteration over the half-open range [begin(), end()) for *this and p.~~

```
int compare(const string_type& s) const
int compare(basic_string_view<value_type> s) const;
```

~~3- Returns: compare(path(s))~~

[Editor's note: Delete paragraph 3 entirely and merge the value_type overload with those above.]

```
int compare(const value_type* s) const
```

-4- ~~Returns~~Effects: Equivalent to: return compare(path(s));.

2943⁽¹⁾. Problematic specification of the wide version of `basic_filebuf::open`

Section: 30.9.2.3 [\[filebuf.members\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2017-03-09 **Last modified:** 2018-08-27

Priority: 2

View all other [issues](#) in [\[filebuf.members\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG [2676](#) specified `basic_filebuf::open(const std::filesystem::path::value_type* s, ios_base::openmode mode)` by simply reusing the specification for the `const char*` overload, but that specification is incorrect for the wide overload: it says that `s` is an NTBS — a null-terminated byte string — which it isn't. Moreover, it specifies that the file is opened as if by calling `fopen(s, modstr)`, but that call is ill-formed if `s` isn't a `const char*`.

[2017-07 Toronto Wed Issue Prioritization]

Priority 2

[2017-11 Albuquerque Wednesday issue processing]

Status to Open; Jonathan to provide wording.

[2018-01-16; Jonathan and Tim Song provide wording]

We'll have to ask the Microsoft guys if "as by a call to fopen" is OK for them. There are paths that can be represented as a wide character string that can't reliably be converted to narrow characters (because they become dependent on the current codepage, or some other Windows nonsense) so they definitely won't use fopen. But as long as they call something that behaves like it (which should allow _fwopen), I think they'll still meet the spirit of the wording.

[2018-08-14; Marshall corrects a grammar nit in the P/R]

The Microsoft guys note that "as by a call to fopen" is OK by them.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4713](#).

1. Edit 30.9.2.3 [\[filebuf.members\]](#) as indicated:

```
basic_filebuf* open(const char* s, ios_base::openmode mode);
basic_filebuf* open(const filesystem::path::value_type* s,
                    ios_base::openmode mode); // wide systems only; see 30.9.1 \[fstream.syn\].
```

~~-?- Requires: s shall point to a NTCTS (20.3.16 [\[defns.ntcts\]](#)).~~

~~-2- Effects: If is_open() != false, returns a null pointer. Otherwise, initializes the filebuf as required. It then opens the file to which s resolves, if possible, as if by a call to fopen with the second argument a file, if possible, whose name is the ntbs s (as if by calling fopen(s, modstr)). The ntbs modstr is determined from mode & ~ios_base::ate as indicated in Table 117. If mode is not some combination of flags shown in the table then the open fails.~~

~~-3- If the open operation succeeds and (mode & ios_base::ate) != 0, positions the file to the end (as if by calling fseek(file, 0, SEEK_END), where file is the pointer returned by calling fopen).(footnote 330)~~

~~-4- If the repositioning operation fails, calls close() and returns a null pointer to indicate failure.~~

~~-5- Returns: this if successful, a null pointer otherwise.~~

[2018-08-23 Batavia Issues processing]

Adopted after changing 'Requires' -> 'Expects' in the P/R.

Proposed resolution:

This wording is relative to [N4713](#).

1. Edit 30.9.2.3 [\[filebuf.members\]](#) as indicated:

```
basic_filebuf* open(const char* s, ios_base::openmode mode);
basic_filebuf* open(const filesystem::path::value_type* s,
                    ios_base::openmode mode); // wide systems only; see 30.9.1 \[fstream.syn\].
```

~~-?- Expects: s shall point to a NTCTS (20.3.16 [\[defns.ntcts\]](#)).~~

~~-2- Effects: If is_open() != false, returns a null pointer. Otherwise, initializes the filebuf as required. It then opens the file to which s resolves, if possible, as if by a call to fopen with the second argument a file, if possible, whose name is the ntbs s (as if by calling fopen(s, modstr)). The ntbs modstr is determined from mode & ~ios_base::ate as indicated in Table 117. If mode is not some combination of flags shown in the table then the open fails.~~

~~-3- If the open operation succeeds and (mode & ios_base::ate) != 0, positions the file to the end (as if by calling fseek(file, 0, SEEK_END), where file is the pointer returned by calling fopen).(footnote 330)~~

~~-4- If the repositioning operation fails, calls close() and returns a null pointer to indicate failure.~~

~~-5- Returns: this if successful, a null pointer otherwise.~~

2960⁽ⁱ⁾. [fund.ts.v3] nonesuch is insufficiently useless

Section: 3.3 [fund.ts.v3::meta] **Status:** [Tentatively Ready](#). **Submitter:** Tim Song **Opened:** 2017-05-08 **Last modified:** 2018-08-27

Priority: 2

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts.v3

The definition of `std::experimental::nonesuch` (with a deleted default constructor, destructor, copy constructor, and copy assignment operator) means that it is an aggregate, which means that it can be initialized from `{}` in contexts where the availability of the destructor is not considered (e.g., overload resolution or a new-expression).

The deleted default constructor also has this effect standing alone, because it doesn't affect the formation of implicit conversion sequences (and hence overload resolution). The net result is ambiguities in situations like:

```
struct such {};  
void f(const such&);  
void f(const nonesuch&);  
f({});
```

For a real-life example of such ambiguity, see [GCC bug 79141](#), involving libstdc++'s internal `__nonesuch` type defined just like the one in the fundamentals TS.

I believe that `nonesuch` would be substantially more useful if the ICS from `{}` is gone. `nonesuch` should have no default constructor (rather than a deleted one), and it shouldn't be an aggregate.

[2017-11-20 Priority set to 2 after discussion on the reflector.]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4617](#).

1. Edit 3.3.1 [fund.ts.v3::meta.type.synop] as indicated, moving the definition of `nonesuch` to 3.3.3 [fund.ts.v3::meta.detect]:

```
//3.3.3 [fund.ts.v3::meta.detect], Detection idiom  
[...]  
  
struct nonesuch{  
    nonesuch() = delete;  
    nonesuch() = delete;  
    nonesuch(nonesuch const&) = delete;  
    void operator=(nonesuch const&) = delete;  
};  
[...]
```

2. Insert at the beginning of 3.3.3 [fund.ts.v3::meta.detect] the following paragraphs:

[Drafting note: The seemingly redundant statement about default and initializer-list constructors is intended to negate the usual leeway for implementations to declare additional member function signatures granted in 20.5.5.5 [\[member.functions\]](#). — end drafting note]

```
struct nonesuch {  
    ~nonesuch() = delete;  
    nonesuch(nonesuch const&) = delete;  
    void operator=(nonesuch const&) = delete;  
};
```

~?- nonesuch has no default constructor (C++14 §[class.ctor]) or initializer-list constructor (C++14 §[dcl.init.list]), and is not an aggregate (C++14 §[dcl.init.aggr]).

[2018-08-23 Batavia Issues processing]

Change C++14 references to C++17, and all the LFTS2 references to LFTS3

Status to Tentatively Ready

Proposed resolution:

This wording is relative to [N4617](#).

1. Edit 3.3.1 [fund.ts.v3::meta.type.synop] as indicated, moving the definition of `nonesuch` to 3.3.3 [fund.ts.v3::meta.detect]:

```
//3.3.3 [fund.ts.v3::meta.detect], Detection idiom  
[...]  
  
struct nonesuch{  
    nonesuch() = delete;  
    nonesuch() = delete;  
    nonesuch(nonesuch const&) = delete;  
    void operator=(nonesuch const&) = delete;  
};  
[...]
```

2. Insert at the beginning of 3.3.3 [fund.ts.v3::meta.detect] the following paragraphs:

[Drafting note: The seemingly redundant statement about default and initializer-list constructors is intended to negate the usual leeway for implementations to declare additional member function signatures granted in 20.5.5.5 [\[member.functions\]](#). — end drafting note]

```
struct nonesuch {  
    ~nonesuch() = delete;  
    nonesuch(nonesuch const&) = delete;  
    void operator=(nonesuch const&) = delete;  
};
```

-?- nonesuch has no default constructor (C++17 §[class.ctor]) or initializer-list constructor (C++17 §[dcl.init.list]), and is not an aggregate (C++17 §[dcl.init.aggr]).

[2995^{\(i\)}](#). `basic_stringbuf` default constructor forbids it from using SSO capacity

Section: 30.8.2.1 [[stringbuf.cons](#)] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2017-07-07 **Last modified:** 2018-06-11

Priority: 3

View other [active issues](#) in [[stringbuf.cons](#)].

View all other [issues](#) in [[stringbuf.cons](#)].

View all issues with [Ready](#) status.

Discussion:

[[stringbuf.cons](#)] says that the default constructor initializes the base class as `basic_streambuf()` which means the all the pointers to the input and output sequences (`pbase`, `eback` etc) are all required to be null.

This means that a `stringbuf` that is implemented in terms of a Small String Optimised `std::basic_string` cannot make use of the string's initial capacity, and so cannot avoid a call to the overflow virtual function even for small writes. In other words, the following assertions must pass:

```
#include <sstream>
#include <cassert>

bool overflowed = false;

struct SB : std::stringbuf
{
    int overflow(int c) {
        assert( pbase() == nullptr );
        overflowed = true;
        return std::stringbuf::overflow(c);
    }
};

int main()
{
    SB sb;
    sb.sputc('1');
    assert(overflowed);
}
```

This is an unnecessary pessimisation. Implementations should be allowed to use the SSO buffer immediately and write to it without calling `overflow`. Libc++ already does this, so is non-conforming.

[2017-07 Toronto Tuesday PM issue prioritization]

Priority 3; is this affected by Peter Sommerlad's [paper P0407R1](#)?

[2018-06 Rapperswil Wednesday issues processing]

Status to Ready

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 30.8.2.1 [[stringbuf.cons](#)] as indicated:

```
explicit basic_stringbuf(
    ios_base::openmode which = ios_base::in | ios_base::out);
```

-1- *Effects:* Constructs an object of class `basic_stringbuf`, initializing the base class with `basic_streambuf()` (30.6.3.1 [[streambuf.cons](#)]), and initializing mode with `which`. It is implementation-defined whether the sequence pointers (`eback()`, `gptr()`, `egptr()`, `pbase()`, `pptr()`, `epptr()`) are initialized to null pointers.

-2- *Postconditions:* `str() == ""`.

[2996^{\(i\)}](#). Missing `rvalue` overloads for `shared_ptr` operations

Section: 23.11.3 [[util.smartptr.shared](#)], 23.11.3.9 [[util.smartptr.shared.cast](#)] **Status:** [Ready](#) **Submitter:** Geoffrey Romer **Opened:** 2017-07-07 **Last modified:** 2018-06-11

Priority: Not Prioritized

View other [active issues](#) in [[util.smartptr.shared](#)].

View all other [issues](#) in [[util.smartptr.shared](#)].

View all issues with [Ready](#) status.

Discussion:

The `shared_ptr` aliasing constructor and the `shared_ptr` casts are specified to take a `shared_ptr` by const reference and construct a new `shared_ptr` that shares ownership with it, and yet they have no corresponding rvalue reference overloads. That results in an unnecessary refcount increment/decrement when those operations are given an rvalue. Rvalue overloads can't be added as a conforming extension because they observably change semantics (but mostly only for code that does unreasonable things like pass an argument by move and then rely on the fact that it's unchanged), and [res.on.arguments]/p1.3 doesn't help because it only applies to rvalue reference parameters.

This issue is related to [P0390R0](#).

[2017-07 Toronto Tuesday PM issue prioritization]

Status LEWG

[2018-06 Rapperswil Monday AM]

Move to Ready; choosing the PR in the issue as opposed to [P0390R0](#) and rebase wording to most recent working draft

Proposed resolution:

This wording is relative to [N4750](#).

1. Edit 23.10.2 [\[memory.syn\]](#), header <memory> synopsis, as indicated:

```
[...]
// 23.11.3.9 \[util.smartptr.shared.cast\], shared_ptr casts
template<class T, class U>
shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
shared\_ptr<T> static\_pointer\_cast\(shared\_ptr<U>&& r\) noexcept;
template<class T, class U>
shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
shared\_ptr<T> dynamic\_pointer\_cast\(shared\_ptr<U>&& r\) noexcept;
template<class T, class U>
shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
shared\_ptr<T> const\_pointer\_cast\(shared\_ptr<U>&& r\) noexcept;
template<class T, class U>
shared_ptr<T> reinterpret_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
shared\_ptr<T> reinterpret\_pointer\_cast\(shared\_ptr<U>&& r\) noexcept;
[...]
```

2. Edit 23.11.3 [\[util.smartptr.shared\]](#), class template `shared_ptr` synopsis, as indicated:

```
template<class T> class shared_ptr {
public:
    [...]
    // 23.11.3.1 \[util.smartptr.shared.const\], constructors
    [...]
    template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
    template<class Y> shared_ptr(const shared_ptr<Y>& r, element_type* p) noexcept;
    template<class Y> shared\_ptr\(shared\_ptr<Y>&& r, element\_type\* p\) noexcept;
    shared_ptr(const shared_ptr& r) noexcept;
    [...]
};
[...]
```

3. Edit 23.11.3.1 [\[util.smartptr.shared.const\]](#) as indicated:

[Drafting note: the `use_count()` postcondition can safely be deleted because it is redundant with the "shares ownership" wording in the *Effects*. — end drafting note]

```
template<class Y> shared_ptr(const shared_ptr<Y>& r, element_type* p) noexcept;
template<class Y> shared\_ptr\(shared\_ptr<Y>&& r, element\_type\* p\) noexcept;
```

-14- *Effects*: Constructs a `shared_ptr` instance that stores `p` and shares ownership with [the initial value of](#) `r`.

-15- *Postconditions*: `get() == p` ~~`&& use_count() == r.use_count()`~~. [For the second overload, `r` is empty and `r.get\(\) == nullptr`.](#)

-16- [Note: To avoid the possibility of a dangling pointer, the user of this constructor must ensure that `p` remains valid at least until the ownership group of `r` is destroyed. — end note]

-17- [Note: This constructor allows creation of an empty `shared_ptr` instance with a non-null stored pointer. — end note]

4. Edit 23.11.3.9 [\[util.smartptr.shared.cast\]](#) as indicated:

```
template<class T, class U>
shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
shared\_ptr<T> static\_pointer\_cast\(shared\_ptr<U>&& r\) noexcept;
```

-1- *Requires*: The expression `static_cast<T*>((U*)nullptr)` shall be well-formed.

-2- *Returns*:

`shared_ptr<T>(R, static_cast<typename shared_ptr<T>::element_type*>(r.get()))`

[, where `R` is `r` for the first overload, and `std::move\(r\)` for the second.](#)

-3- [Note: The seemingly equivalent expression `shared_ptr<T>(static_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

```
template<class T, class U>
    shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> dynamic_pointer_cast(shared_ptr<U>&& r) noexcept;
```

-4- *Requires:* The expression `dynamic_cast<T*>((U*)nullptr)` shall be well-formed. The expression `dynamic_cast<typename shared_ptr<T>::element_type*>(r.get())` shall be well formed and shall have well-defined behavior.

-5- *Returns:*

(5.1) — When `dynamic_cast<typename shared_ptr<T>::element_type*>(r.get())` returns a non-null value `p`, `shared_ptr<T>(R, p)`, where `R` is `r` for the first overload, and `std::move(r)` for the second.

(5.2) — Otherwise, `shared_ptr<T>()`.

-6- [Note: The seemingly equivalent expression `shared_ptr<T>(dynamic_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

```
template<class T, class U>
    shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> const_pointer_cast(shared_ptr<U>&& r) noexcept;
```

-7- *Requires:* The expression `const_cast<T*>((U*)nullptr)` shall be well-formed.

-8- *Returns:*

`shared_ptr<T>(R, const_cast<typename shared_ptr<T>::element_type*>(r.get()))`

, where `R` is `r` for the first overload, and `std::move(r)` for the second.

-9- [Note: The seemingly equivalent expression `shared_ptr<T>(const_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

```
template<class T, class U>
    shared_ptr<T> reinterpret_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> reinterpret_pointer_cast(shared_ptr<U>&& r) noexcept;
```

-10- *Requires:* The expression `reinterpret_cast<T*>((U*)nullptr)` shall be well-formed.

-11- *Returns:*

`shared_ptr<T>(R, reinterpret_cast<typename shared_ptr<T>::element_type*>(r.get()))`

, where `R` is `r` for the first overload, and `std::move(r)` for the second.

-12- [Note: The seemingly equivalent expression `shared_ptr<T>(reinterpret_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

3008⁽¹⁾. make_shared (sub)object destruction semantics are not specified

Section: 23.11.3.6 [util.smartptr.shared.create] **Status:** [Tentatively Ready](#) **Submitter:** Glen Joseph Fernandes **Opened:** 2017-08-06 **Last modified:** 2018-08-27

Priority: 2

View all other [issues](#) in [util.smartptr.shared.create].

View all issues with [Tentatively Ready](#) status.

Discussion:

The remarks for the `make_shared` and `allocate_shared` functions do not specify how the objects managed by the returned `shared_ptr` are destroyed. It is implied that when objects are constructed via a placement new expression, they are destroyed by calling the destructor, and that when objects are constructed via an allocator, they are destroyed using that allocator. This should be explicitly specified.

[2017-11 Albuquerque Wednesday night issues processing]

Priority set to 2

Previous resolution [SUPERSEDED]:

This resolution is relative to [N4687](#).

1. Edit 23.11.3.6 [util.smartptr.shared.create] as indicated:

```
template<class T, ...>
    shared_ptr<T> make_shared(args);
template<class T, class A, ...>
    shared_ptr<T> allocate_shared(const A& a, args);
```

[...]

-7- Remarks:

[...]

(7.9) — When the lifetime of the object managed by the return value ends, or when the initialization of an array element throws an exception, the initialized elements should be destroyed in the reverse order of their construction.

(7.2) — When a (sub)object of a non-array type U that was initialized by `make_shared` is to be destroyed, it shall be destroyed via the expression `pv->~U()` where `pv` points to that object of type U.

(7.2) — When a (sub)object of a non-array type U that was initialized by `allocate_shared` is to be destroyed, it shall be destroyed via the expression `allocator_traits<A2>::destroy(a2, _pv)` where `pv` points to that object of type `cv-unqualified U` and `a2` of type `A2` is a rebound copy of the allocator `a` passed to `allocate_shared` such that its value type is `remove_cv_t<U>`.

[2018-06 Rapperswil Wednesday night issues processing]

CC: what is "of type `cv-unqualified U`" and "`remove_cv_t<U>`" about?

DK: again, it isn't new wording; it is in p 7.5.2

JW: but none of the words use "of type `cv-unqualified U`"

CT: so we should also use `remove_cv_t<U>` instead?

JW: I would like to talk to Glen

FB: does anybody know how it works for an array of arrays? It seems to cover the case

JW: we could leave it vague as it is now or specify it to exactly what it does

DK: I think we should split the thing into two parts and start with definitions

DK: ACTION I can refactor the wording

MC: there was a fairly long message thread when we talked about this

Daniel comments and improves wording:

The currently allocator requirements support only the construction of `cv-unqualified` object types (See Table 30 type `c` and pointer variable `c` as well as Table 31 expressions "`a.construct(c, args)`" and "`a.destroy(c)`"), therefore a conforming implementation needs to effectively construct an object pointer that holds an object of type `remove_cv_t<U>` and similarly destroy such an object. Albeit it seems to be an artificial restriction to construct and destroy only non-`cv`-qualified object types, this is, if any, a different issue. But given this current state, the wording for `allocate_shared` needs to make a special wording dance via `remove_cv_t<U>`. For construct the existing wording prevents to speak about that detail by using the more indirect phrase "where `pv` points to storage suitable to hold an object of type `U`", but since object types `U` and `const U` have exactly the same storage and alignment requirements, this sentence is correct for `remove_cv_t<U>` as well.

[2018-08-23 Batavia Issues processing]

Status to Tentatively Ready.

Proposed resolution:

This resolution is relative to [N4750](#).

1. Edit 23.11.3.6 [\[util.smartptr.shared.create\]](#) as indicated:

```
template<class T, ...>
shared_ptr<T> make_shared(args);
template<class T, class A, ...>
shared_ptr<T> allocate_shared(const A& a, args);
```

[...]

-7- Remarks:

[...]

(7.9) — When the lifetime of the object managed by the return value ends, or when the initialization of an array element throws an exception, the initialized elements are destroyed in the reverse order of their original construction.

(7.2) — When a (sub)object of a non-array type U that was initialized by `make_shared` is to be destroyed, it is destroyed via the expression `pv->~U()` where `pv` points to that object of type U.

(7.2) — When a (sub)object of a non-array type U that was initialized by `allocate_shared` is to be destroyed, it is destroyed via the expression `allocator_traits<A2>::destroy(a2, pv)` where `pv` points to that object of type `remove_cv_t<U>` and `a2` of type `A2` is a rebound copy of the allocator `a` passed to `allocate_shared` such that its value type is `remove_cv_t<U>`.

3025⁽ⁱ⁾. Map-like container deduction guides should use `pair<Key, T>`, not `pair<const Key, T>`

Section: 26.4.4.1 [\[map.overview\]](#), 26.4.5.1 [\[multimap.overview\]](#), 26.5.4.1 [\[unord.map.overview\]](#), 26.5.5.1 [\[unord.multimap.overview\]](#) Status: [Tentatively Ready](#)
Submitter: Ville Voutilainen Opened: 2017-10-08 Last modified: 2018-08-27

Priority: 2

View all other [issues](#) in [\[map.overview\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

With the deduction guides as specified currently, code like this doesn't work:

```
map m{pair{1, 1}, {2, 2}, {3, 3}};
```

Same problem occurs with `multimap`, `unordered_map` and `unordered_multimap`. The problem is in deduction guides like

```
template<class Key, class T, class Compare = less<Key>,
        class Allocator = allocator<pair<const Key, T>>>
map(initializer_list<pair<const Key, T>>, Compare = Compare(),
    Allocator = Allocator()) -> map<Key, T, Compare, Allocator>;
```

The `pair<const Key, T>` is not matched by a `pair<int, int>`, because `int` can't match a `const Key`. Dropping the `const` from the parameter of the deduction guide makes it work with no loss of functionality.

[2017-11-03, Zhihao Yuan comments]

The fix described [here](#) prevents

```
std::map m2{m0.begin(), m0.end()};
```

from falling back to direct-non-list-initialization. Treating a uniform initialization with >1 clauses of the same un-cvref type as `std::initializer_list` is the only consistent interpretation I found so far.

[2017-11 Albuquerque Wednesday night issues processing]

Priority set to 2

[2018-08-23 Batavia Issues processing]

Status to Tentatively Ready

Proposed resolution:

This wording is relative to [N4687](#).

1. Change 26.4.4.1 [\[map.overview\]](#) p3, class template `map` synopsis, as indicated:

```
[...]
template<class Key, class T, class Compare = less<Key>,
        class Allocator = allocator<pair<const Key, T>>>
map(initializer_list<pair<const Key, T>>, Compare = Compare(), Allocator = Allocator())
-> map<Key, T, Compare, Allocator>;
[...]
template<class Key, class T, class Allocator>
map(initializer_list<pair<const Key, T>>, Allocator) -> map<Key, T, less<Key>, Allocator>;
[...]
```

2. Change 26.4.5.1 [\[multimap.overview\]](#) p3, class template `multimap` synopsis, as indicated:

```
[...]
template<class Key, class T, class Compare = less<Key>,
        class Allocator = allocator<pair<const Key, T>>>
multimap(initializer_list<pair<const Key, T>>, Compare = Compare(), Allocator = Allocator())
-> multimap<Key, T, Compare, Allocator>;
[...]
template<class Key, class T, class Allocator>
multimap(initializer_list<pair<const Key, T>>, Allocator)
-> multimap<Key, T, less<Key>, Allocator>;
[...]
```

3. Change 26.5.4.1 [\[unord.map.overview\]](#) p3, class template `unordered_map` synopsis, as indicated:

```
[...]
template<class Key, class T, class Hash = hash<Key>,
        class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
unordered_map(initializer_list<pair<const Key, T>>, typename see below::size_type = see below, Hash = Hash(),
    Pred = Pred(), Allocator = Allocator())
-> unordered_map<Key, T, Hash, Pred, Allocator>;
[...]
template<class Key, class T, typename Allocator>
unordered_map(initializer_list<pair<const Key, T>>, typename see below::size_type,
    Allocator)
-> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, typename Allocator>
unordered_map(initializer_list<pair<const Key, T>>, Allocator)
-> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Hash, class Allocator>
unordered_map(initializer_list<pair<const Key, T>>, typename see below::size_type, Hash,
    Allocator)
-> unordered_map<Key, T, Hash, equal_to<Key>, Allocator>;
[...]
```

4. Change 26.5.5.1 [\[unord.multimap.overview\]](#) p3, class template `unordered_multimap` synopsis, as indicated:

```
[...]
template<class Key, class T, class Hash = hash<Key>,
        class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
unordered_multimap(initializer_list<pair<const Key, T>>,
    typename see below::size_type = see below,
    Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multimap<Key, T, Hash, Pred, Allocator>;
[...]
template<class Key, class T, typename Allocator>
```

```

unordered_multimap(initializer_list<pair<const Key, T>>, typename see below::size_type,
                  Allocator)
-> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, typename Allocator>
unordered_multimap(initializer_list<pair<const Key, T>>, Allocator)
-> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Hash, class Allocator>
unordered_multimap(initializer_list<pair<const Key, T>>, typename see below::size_type,
                  Hash, Allocator)
-> unordered_multimap<Key, T, Hash, equal_to<Key>, Allocator>;
[...]
```

3031⁽ⁱ⁾. Algorithms and predicates with non-const reference arguments

Section: 28.7 [[alg.sorting](#)] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2017-11-08 **Last modified:** 2018-08-27

Priority: 2

View all other [issues](#) in [alg.sorting].

View all issues with [Tentatively Ready](#) status.

Discussion:

This doesn't compile with any major implementation:

```

int i[1] = { };
std::stable_sort(i, i, [](int& x, int& y) { return x < y; });
```

The problem is that the Compare expects non-const references. We say "It is assumed that comp will not apply any non-constant function through the dereferenced iterator" But that isn't sufficient to forbid the example.

My first thought was to modify [alg.sorting] to make the Compare requirements use `comp(as_const(x), as_const(x))` but that would get very verbose to add to every expression using `comp`.

[2017-11 Albuquerque Wednesday night issues processing]

Priority set to 2; Jonathan to improve the statement of the problem.

[2018-02 David Jones provided this [truly awful example](#):]

```

#include <algorithm>
#include <iostream>
#include <vector>

struct Base {
    Base(int value) : v(value) {}
    friend bool operator<(const Base& l, const Base& r) { return l.v < r.v; }
    int v;
};

struct Derived : public Base {
    using Base::Base;
    bool operator<(const Derived& o) /* no const here */ { return v > o.v; }
};

int main(void) {
    std::vector<Base> b = {{1}, {5}, {0}, {3}};
    std::vector<Derived> d = {{0}, {1}, {3}, {5}};

    std::cout << std::lower_bound(d.begin(), d.end(), 4)->v << std::endl;

    std::sort(b.begin(), b.end());
    for (const auto &x : b) std::cout << x.v << " ";
    std::cout << std::endl;

    std::sort(d.begin(), d.end());
    for (const auto &x : d) std::cout << x.v << " ";
    std::cout << std::endl;
}

libc++:
=====
$ bin/clang++ -std=c++11 -stdlib=libc++ tmp/ex.cc && ./a.out
5
0 1 3 5
0 1 3 5
=====

libstdc++:
=====
$ bin/clang++ -std=c++11 -stdlib=libstdc++ tmp/ex.cc && ./a.out
0
0 1 3 5
5 3 1 0
=====
```

[2018-08 Batavia Monday issue discussion]

Tim to provide wording; status to 'Open'

[2018-08-20, Tim adds P/R based on Batavia discussion.]

Similar to the Ranges TS design, the P/R below requires Predicate, BinaryPredicate, and Compare to accept all mixes of const and non-const arguments.

[2018-08-23 Batavia Issues processing]

Status to Tentatively Ready after minor wording nit (corrected in place)

Proposed resolution:

This wording is relative to [N4762](#).

1. Edit 28.3 [\[algorithms.requirements\]](#) p6-7 as indicated:

-6- The Predicate parameter is used whenever an algorithm expects a function object (23.14 [\[function.objects\]](#)) that, when applied to the result of dereferencing the corresponding iterator, returns a value testable as true. In other words, if an algorithm takes Predicate pred as its argument and first as its iterator argument **with value type T**, it should work correctly in the construct pred(*first) contextually converted to bool (7 [\[conv\]](#)). The function object pred shall not apply any non-constant function through the dereferenced iterator. **Given a glvalue u of type (possibly const) T that designates the same object as *first, pred(u) shall be a valid expression that is equal to pred(*first).**

-7- The BinaryPredicate parameter is used whenever an algorithm expects a function object that when applied to the result of dereferencing two corresponding iterators or to dereferencing an iterator and type T when T is part of the signature returns a value testable as true. In other words, if an algorithm takes BinaryPredicate binary_pred as its argument and first1 and first2 as its iterator arguments **with respective value types T1 and T2**, it should work correctly in the construct binary_pred(*first1, *first2) contextually converted to bool (7 [\[conv\]](#)). **Unless otherwise specified**, BinaryPredicate always takes the first iterator's value_type as its first argument, that is, in those cases when T value is part of the signature, it should work correctly in the construct binary_pred(*first1, value) contextually converted to bool (7 [\[conv\]](#)). binary_pred shall not apply any non-constant function through the dereferenced iterators. **Given a glvalue u of type (possibly const) T1 that designates the same object as *first1, and a glvalue v of type (possibly const) T2 that designates the same object as *first2, binary_pred(u, *first2), binary_pred(*first1, v), and binary_pred(u, v) shall each be a valid expression that is equal to binary_pred(*first1, *first2), and binary_pred(u, value) shall be a valid expression that is equal to binary_pred(*first1, value).**

2. Edit 28.7 [\[alg.sorting\]](#) p2 as indicated:

Compare is a function object type (23.14 [\[function.objects\]](#)) **that meets the requirements for a template parameter named BinaryPredicate (28.3 [\[algorithms.requirements\]](#)).** The return value of the function call operation applied to an object of type Compare, when contextually converted to bool (7 [\[conv\]](#)), yields true if the first argument of the call is less than the second, and false otherwise. Compare comp is used throughout for algorithms assuming an ordering relation. ~~It is assumed that comp will not apply any non-constant function through the dereferenced iterator.~~

[3037^{\(i\)}](#). polymorphic_allocator and incomplete types

Section: 23.12.3 [\[mem.poly.allocator.class\]](#) Status: [Tentatively Ready](#) Submitter: Casey Carter Opened: 2017-11-15 Last modified: 2018-08-27

Priority: 2

View other [active issues](#) in [\[mem.poly.allocator.class\]](#).

View all other [issues](#) in [\[mem.poly.allocator.class\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

polymorphic_allocator can trivially support the allocator completeness requirements (20.5.3.5.1 [\[allocator.requirements.completeness\]](#)) just as does the default allocator. Doing so imposes no implementation burden, and enables pmr::forward_list, pmr::list, and pmr::vector to support incomplete types as do the non-pmr equivalents.

[2018-01; Priority set to 2 after mailing list discussion]

[2018-08-23 Batavia Issues processing]

Status to Tentatively Ready

Proposed resolution:

Wording relative to [N4700](#).

1. Add a new paragraph in 23.12.3 [\[mem.poly.allocator.class\]](#) after para 1:

1 A specialization of class template pmr::polymorphic_allocator conforms to the Allocator requirements [...]

-?- All specializations of class template pmr::polymorphic_allocator satisfy the allocator completeness requirements (20.5.3.5.1 [\[allocator.requirements.completeness\]](#)).

[3038^{\(i\)}](#). polymorphic_allocator::allocate should not allow integer overflow to create vulnerabilities

Section: 23.12.3.2 [\[mem.poly.allocator.mem\]](#) Status: [Tentatively Ready](#) Submitter: Billy O'Neal III Opened: 2017-11-16 Last modified: 2018-08-27

Priority: 2

View other [active issues](#) in [mem.poly.allocator.mem].

View all other [issues](#) in [mem.poly.allocator.mem].

View all issues with [Tentatively Ready](#) status.

Discussion:

At the moment polymorphic_allocator is specified to do `sizeof(T) * n` directly; this may allow an attacker to cause this calculation to overflow, resulting in `allocate()` not meeting its postcondition of returning a buffer suitable to store `n` copies of `T`; this is a common bug described in [CWE-190](#).

Making this into a saturating multiply should be sufficient to avoid this problem; any memory_resource underneath polymorphic_allocator is going to have to throw `bad_alloc` (or another exception) for a request of `SIZE_MAX`.

(There's also a minor editorial thing here that *Returns* should be *Effects*)

[2018-06 Rapperswil Thursday issues processing]

Consensus was that the overflow should be detected and an exception thrown rather than leaving that to the underlying memory resource. Billy to reword, and then get feedback on the reflector. Status to Open.

Previous resolution [SUPERSEDED]:

Wording relative to [N4700](#).

1. Edit 23.12.3.2 [\[mem.poly.allocator.mem\]](#) as indicated:

```
    Tp* allocate(size_t n);
```

-1- ~~Returns~~ **Effects**: Equivalent to

```
    return static_cast<Tp*>(memory_rsrc->allocate(SIZE_MAX / sizeof(Tp) < n ? SIZE_MAX : n * sizeof(Tp), alignof(Tp)))
```

[2018-08-23 Batavia Issues processing]

Status to Tentatively Ready with updated wording

Previous resolution [SUPERSEDED]:

Wording relative to [N4762](#).

1. Edit 23.12.3.2 [\[mem.poly.allocator.mem\]](#) as indicated:

```
    Tp* allocate(size_t n);
```

-1- **Effects**: **If `SIZE_MAX / sizeof(Tp) < n`, throws length error, then** ~~Equivalent~~ to:

```
    return static_cast<Tp*>(memory_rsrc->allocate(n * sizeof(Tp), alignof(Tp)));
```

Proposed resolution:

Wording relative to [N4762](#).

1. Edit 23.12.3.2 [\[mem.poly.allocator.mem\]](#) as indicated:

```
    Tp* allocate(size_t n);
```

-1- **Effects**: **If `SIZE_MAX / sizeof(Tp) < n`, throws length error. Otherwise** ~~Equivalent~~ to:

```
    return static_cast<Tp*>(memory_rsrc->allocate(n * sizeof(Tp), alignof(Tp)));
```

[3054^{\(i\)}](#). uninitialized_copy appears to not be able to meet its exception-safety guarantee

Section: 23.10.11.4 [\[uninitialized.copy\]](#) Status: [Ready](#) Submitter: Jon Cohen Opened: 2018-01-24 Last modified: 2018-08-10

Priority: 2

View all other [issues](#) in [uninitialized.copy].

View all issues with [Ready](#) status.

Discussion:

I believe that `uninitialized_copy` is unable to meet its exception-safety guarantee in the presence of throwing move constructors:

23.10.11 [\[specialized.algorithms\]](#)/1 has two statements of note for the specialized algorithms such as `uninitialized_copy`:

- the provided iterators satisfy the `InputIterator` requirements (27.2.3 [\[input.iterators\]](#))
- if an exception is thrown during the algorithm then there are no effects

Suppose we have an input iterator `Iter`. Then `std::move_iterator<Iter>` appears to also be an input iterator. Notably, it still satisfies that `(void)*a, *a` is equivalent to `*a` for move iterator `a` since the dereference only forms an rvalue reference, it doesn't actually perform the move operation (27.2.3 [\[input.iterators\]](#) Table 95 — "Input iterator requirements").

Suppose also that we have a type `T` whose move constructor can throw, a range of `T`'s `[tbegin, tend)`, and a pointer to an uninitialized buffer of `T`'s `buf`. Then `std::uninitialized_copy(std::make_move_iterator(tbegin), std::make_move_iterator(tend), buf)` can't possibly satisfy the property that it has no effects if one of the moves throws — we'll have a `T` left in a moved-from state with no way of recovering.

See [here](#) for an example in code.

It seems like the correct specification for `uninitialized_copy` should be that if `InputIterator`'s `operator*` returns an rvalue reference and `InputIterator::value_type`'s move constructor is not marked `noexcept`, then `uninitialized_copy` will leave the objects in the underlying range in a valid but unspecified state.

[2018-01-24, Casey comments and provides wording]

This issue points out a particular hole in the "...if an exception is thrown in the following algorithms there are no effects." wording for the "uninitialized" memory algorithms (23.10.11 [\[specialized.algorithms\]](#)/1) and suggests a PR to patch over said hole. The true problem here is that "no effects" is not and never has been implementable. For example, "`first != last`" may have observable effects that an implementation is required to somehow reverse if some later operation throws an exception.

Rather than finding problem case after problem case and applying individual patches, we should fix the root cause. If we alter the problematic sentence from [\[specialized.algorithms\]](#)/1 we can fix the issue once and for all and have implementable algorithms.

[2018-02-05, Priority set to 2 after mailing list discussion]

[2018-06 Rapperswil Thursday issues processing]

Status to Ready

Proposed resolution:

This wording is relative to [N4713](#).

1. Modify 23.10.11 [\[specialized.algorithms\]](#) as indicated:

-1- [...]

Unless otherwise specified, if an exception is thrown in the following algorithms [objects constructed by a placement new-expression \(8.5.2.4 \[expr.new\]\) are destroyed in an unspecified order before allowing the exception to propagate](#) ~~there are no effects.~~

2. Modify 23.10.11.5 [\[uninitialized.move\]](#) as indicated (The removed paragraphs are now unnecessary):

```
template<class InputIterator, class ForwardIterator>
    ForwardIterator uninitialized_move(InputIterator first, InputIterator last,
                                     ForwardIterator result);
```

[...]

~~-2- Remarks: If an exception is thrown, some objects in the range `[first, last)` are left in a valid but unspecified state.~~

```
template<class InputIterator, class Size, class ForwardIterator>
    pair<InputIterator, ForwardIterator>
    uninitialized_move_n(InputIterator first, Size n, ForwardIterator result);
```

[...]

~~-4- Remarks: If an exception is thrown, some objects in the range `[first, std::next(first, n))` are left in a valid but unspecified state.~~

[3065^{\(i\)}](#). LWG 2989 missed that all path's other operators should be hidden friends as well

Section: 30.11.7.7 [\[fs.path.nonmember\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Billy O'Neal III **Opened:** 2018-02-13 **Last modified:** 2018-08-27

Priority: 2

View all issues with [Tentatively Ready](#) status.

Discussion:

Consider the following program:

```
// See godbolt link
#include <assert.h>
#include <string>
#include <filesystem>

using namespace std;
using namespace std::filesystem;

int main() {
    bool b = L"a//b" == std::string("a/b");
    assert(b); // passes. What?!
    return b;
}
```

L"a" gets converted into a path, and the string gets converted into a path, and then those paths are compared for equality. But path equality comparison doesn't work anything like string equality comparison, leading to surprises.

path's other operators should be made hidden friends as well, so that one side or the other of a given operator is of type path before those conversions apply.

[2018-02-20, Priority set to 2 after mailing list discussion]

[2018-08-23 Batavia Issues processing]

Status to Tentatively Ready

Proposed resolution:

This wording is relative to [N4713](#).

All drafting notes from LWG [2989](#) apply here too.

1. Modify 30.11.5 [\[fs.filesystem.syn\]](#), header <filesystem> synopsis, as indicated:

```
[...]
// 30.11.7.7 \[fs.path.nonmember\], path non-member functions
void swap(path& lhs, path& rhs) noexcept;
size_t hash_value(const path& p) noexcept;

bool operator==(const path& lhs, const path& rhs) noexcept;
bool operator!=(const path& lhs, const path& rhs) noexcept;
bool operator<(const path& lhs, const path& rhs) noexcept;
bool operator<=(const path& lhs, const path& rhs) noexcept;
bool operator>(const path& lhs, const path& rhs) noexcept;
bool operator>=(const path& lhs, const path& rhs) noexcept;
bool operator/=(const path& lhs, const path& rhs) noexcept;

path operator/ (const path& lhs, const path& rhs);
[...]
```

2. Modify 30.11.7.7 [\[fs.path.nonmember\]](#) as indicated:

```
[...]
friend bool operator< (const path& lhs, const path& rhs) noexcept;
[...]
```

```
[...]
friend bool operator<=(const path& lhs, const path& rhs) noexcept;
[...]
```

```
[...]
friend bool operator> (const path& lhs, const path& rhs) noexcept;
[...]
```

```
[...]
friend bool operator>=(const path& lhs, const path& rhs) noexcept;
[...]
```

```
[...]
friend bool operator==(const path& lhs, const path& rhs) noexcept;
[...]
```

```
[...]
friend bool operator!=(const path& lhs, const path& rhs) noexcept;
[...]
```

```
[...]
friend path operator/ (const path& lhs, const path& rhs);
[...]
```

3. Modify 30.11.7 [\[fs.class.path\]](#), class path synopsis, as indicated:

```
class path {
public:
    [...]
    // 30.11.7.4.5 \[fs.path.modifiers\], modifiers
    [...]

    // 30.11.7.7 \[fs.path.nonmember\], non-member operators
    friend bool operator< (const path& lhs, const path& rhs) noexcept;
    friend bool operator<=(const path& lhs, const path& rhs) noexcept;
    friend bool operator> (const path& lhs, const path& rhs) noexcept;
    friend bool operator>=(const path& lhs, const path& rhs) noexcept;
    friend bool operator==(const path& lhs, const path& rhs) noexcept;
    friend bool operator!=(const path& lhs, const path& rhs) noexcept;

    friend path operator/ (const path& lhs, const path& rhs);

    // 30.11.7.4.6 \[fs.path.native.obs\], native format observers
    [...]
};
```

[3096^{\(i\)}](#). path::lexically_relative is confused by trailing slashes

Section: 30.11.7.4.11 [\[fs.path.gen\]](#) Status: [Tentatively Ready](#) Submitter: Jonathan Wakely Opened: 2018-04-04 Last modified: 2018-08-27

Priority: 2

View other [active issues](#) in [fs.path.gen].

View all other [issues](#) in [fs.path.gen].

View all issues with [Tentatively Ready](#) status.

Discussion:

filesystem::proximate("/dir", "/dir/") returns "." when "/dir" exists, and "." otherwise. It should always be "." because whether it exists shouldn't matter.

The problem is in `filesystem::path::lexically_relative`, as shown by:

```
path("/dir").lexically_relative("/dir/."); // yields ""
path("/dir").lexically_relative("/dir/"); // yields ".."
```

The two calls should yield the same result, and when iteration of a path with a trailing slash gave "." as the final element they did yield the same result. In the final C++17 spec the trailing slash produces an empty filename in the iteration sequence, and `lexically_relative` doesn't handle that correctly.

[2018-04-10, Jonathan comments]

There are more inconsistencies with paths that are "obviously" equivalent to the human reader:

```
path("a/b/c").lexically_relative("a/b/c") // yields "."
path("a/b/c").lexically_relative("a/b/c/") // yields ".."
path("a/b/c").lexically_relative("a/b/c/.") // yields ""
path("a/b/c/").lexically_relative("a/b/c") // yields ""
path("a/b/c/.").lexically_relative("a/b/c") // yields "."
path("a/b/c/.").lexically_relative("a/b/c/") // yields "..".
```

I think the right solution is:

1. when counting `[b, base.end())` in bullet (4.2) handle empty filename elements (which can only occur as the last element, due to a trailing slash) equivalently to dot elements; and
2. add a new condition for the case where `n == 0` and `[a, end())` contains no non-empty elements, i.e. the paths are equivalent except for final dots or a final slash, which don't introduce any relative difference between the paths.

[2018-06-18 after reflector discussion]

Priority set to 2

[2018-08-23 Batavia Issues processing]

Status to Tentatively Ready

Proposed resolution:

This wording is relative to [N4727](#).

1. Edit 30.11.7.4.11 [\[fs.path.gen\]](#) as indicated:

```
path lexically_relative(const path& base) const;
```

-3- *Returns:* *this made relative to base. Does not resolve (30.11.7 [\[fs.class.path\]](#)) symlinks. Does not first normalize (30.11.7.1 [\[fs.path.generic\]](#)) *this or base.

-4- *Effects:* If `root_name() != base.root_name()` is true or `is_absolute() != base.is_absolute()` is true or `!has_root_directory() && base.has_root_directory()` is true, returns `path()`. Determines the first mismatched element of *this and base as if by:

```
auto [a, b] = mismatch(begin(), end(), base.begin(), base.end());
```

Then,

(4.1) — if `a == end()` and `b == base.end()`, returns `path(".");` otherwise

(4.2) — let `n` be the number of *filename* elements in `[b, base.end())` that are not dot or dot-dot **or empty**, minus the number that are dot-dot. If `n < 0`, returns `path()`; otherwise

(4.2) — if `n == 0` and `(a == end() || a->empty())`, returns `path(".");` otherwise

(4.3) — returns an object of class `path` that is default-constructed, followed by [...]

[3116^{\(i\)}](#). **OUTERMOST_ALLOC_TRAITS** needs **remove_reference_t**

Section: 23.13.4 [\[allocator.adaptor.members\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2018-06-04 **Last modified:** 2018-06-11

Priority: 0

View all other [issues](#) in [\[allocator.adaptor.members\]](#).

View all issues with [Ready](#) status.

Discussion:

`OUTERMOST_ALLOC_TRAITS(x)` is currently defined in 23.13.4 [\[allocator.adaptor.members\]](#)p1 as `allocator_traits<decltype(OUTERMOST(x))>`. However, `OUTERMOST(x)`, as defined and used in this subclause, is an lvalue for which `decltype` produces an lvalue reference. That referenceness needs to be removed before the type can be used with `allocator_traits`.

While we are here, the current wording for `OUTERMOST` uses the imprecise "if `x` does not have an `outer_allocator()` member function". What we meant to check is the validity of the expression `x.outer_allocator()`, not whether `x` has some (possibly ambiguous and/or inaccessible) member function named `outer_allocator`.

[2018-06 Rapperswil Thursday issues processing]

Status to Ready

Proposed resolution:

This wording is relative to [N4750](#).

[Drafting note: The subclause only uses `OUTERMOST_ALLOC_TRAITS(*this)` and only in non-const member functions, so the result is also non-const. Thus, `remove_reference_t` is sufficient; there's no need to further remove cv-qualification. — end drafting note]

1. Modify 23.13.4 [\[allocator.adaptor.members\]](#)p1 as indicated:

-1- In the construct member functions, `OUTERMOST(x)` is ~~`x` if `x` does not have an `outer_allocator()` member function and `OUTERMOST(x.outer_allocator())` if the expression `x.outer_allocator()` is valid (17.9.2 [\[temp.deduct\]](#)) and `x`~~ otherwise; `OUTERMOST_ALLOC_TRAITS(x)` is `allocator_traits<remove_reference_t<decltype(OUTERMOST(x))>>`. [Note: [...] — end note]

3122⁽ⁱ⁾. `__cpp_lib_chrono_udls` was accidentally dropped

Section: 21.3.1 [\[support.limits.general\]](#) Status: [Tentatively Ready](#) Submitter: Stephan T. Lavavej Opened: 2018-06-14 Last modified: 2018-06-24

Priority: 0

View other [active issues](#) in [\[support.limits.general\]](#).

View all other [issues](#) in [\[support.limits.general\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

Between [P0941R0](#) and [P0941R1/P0941R2](#), the feature-test macro `__cpp_lib_chrono_udls` was dropped. It wasn't mentioned in the changelog, and Jonathan Wakely and I believe that this was unintentional.

[2018-06-23 Moved to Tentatively Ready after 5 positive votes on `c++std-lib`.]

Proposed resolution:

This wording is relative to the post-Rapperswil 2018 working draft.

In 21.3.1 [\[support.limits.general\]](#), "Table ??? - Standard library feature-test macros", add the following row:

Table ??? — Standard library feature-test macros

Macro name	Value	Headers
[...]		
<code>__cpp_lib_chrono_udls</code>	201304L	<code><chrono></code>
[...]		

3127⁽ⁱ⁾. `basic_osyncstream::rdbuf` needs a `const_cast`

Section: 30.10.3.1 [\[syncstream.osyncstream.overview\]](#) Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2018-06-29 Last modified: 2018-07-20

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

The current specification of `basic_osyncstream::rdbuf()` is

```
syncbuf_type* rdbuf() const noexcept { return &sb; }
```

This is ill-formed because the exposition-only member `sb` is `const` inside this `const` member function, but the return type is a pointer to non-const `syncbuf_type`. It needs to cast away the constness, consistent with the other streams with embedded stream buffers (such as `string` and `file streams`).

[2018-07-20 Status set to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4750](#).

1. Change 30.10.3.1 [\[syncstream.osyncstream.overview\]](#), class template `basic_osyncstream` synopsis, as indicated:

```
namespace std {
    template<class charT, class traits, class Allocator>
    class basic_osyncstream : public basic_ostream<charT, traits> {
    public:
        [...]

        // 30.10.3.4 \[syncstream.osyncstream.members\], member functions
        void emit();
        streambuf_type* get_wrapped() const noexcept;
        syncbuf_type* rdbuf() const noexcept { return const\_cast<syncbuf\_type\*>\(&sb\); }
    };
```

```
    [...]
};
}
```

[3128^{\(0\)}](#). `strstream::rdbuf` needs a `const_cast`

Section: D.8.5.3 [\[depr.strstream.oper\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2018-06-30 **Last modified:** 2018-07-20

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

`strstream::rdbuf` has the same issue with a missing `const_cast` on `&sb`.

Somewhat amusingly, `istrstream::rdbuf` and `ostrstream::rdbuf` got this right, but each with a different style (see issue [252](#)).

[2018-07-20 Status to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4750](#).

1. Change D.8.5.3 [\[depr.strstream.oper\]](#) p1 as indicated:

```
strstreambuf* rdbuf() const;
```

-1- Returns: `const_cast<strstreambuf*>(&sb)`.

[3129^{\(0\)}](#). `regex_token_iterator` constructor uses wrong pointer arithmetic

Section: 31.12.2.1 [\[re.tokiter.cnstr\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2018-06-30 **Last modified:** 2018-07-20

Priority: 0

View all other issues in `[re.tokiter.cnstr]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

The specification of `regex_token_iterator` for the overload taking a `const int (&submatches)[N]` uses the range `[&submatches, &submatches + N)`. This is obviously incorrect; we want to perform pointer arithmetic on a pointer to the first element of that array, not a pointer to the whole array.

[2018-07-20 Status to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4750](#).

1. Change 31.12.2.1 [\[re.tokiter.cnstr\]](#) p3 as indicated:

```
regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                    const regex_type& re,
                    int submatch = 0,
                    regex_constants::match_flag_type m = regex_constants::match_default);
```

```
regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                    const regex_type& re,
                    const vector<int>& submatches,
                    regex_constants::match_flag_type m = regex_constants::match_default);
```

```
regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                    const regex_type& re,
                    initializer_list<int> submatches,
                    regex_constants::match_flag_type m = regex_constants::match_default);
```

```
template<size_t N>
regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                    const regex_type& re,
                    const int (&submatches)[N],
                    regex_constants::match_flag_type m = regex_constants::match_default);
```

-2- Requires: [...]

-3- Effects: The first constructor initializes the member `subs` to hold the single value `submatch`. ~~The second constructor initializes the member `subs` to hold a copy of the argument `submatches`. The second, third and fourth constructors initialize the member `subs` to hold a copy of the sequence of integer values pointed to by the iterator range `[submatches.begin(), submatches.end())` and `[&submatches, &submatches + N)`, respectively.~~ `[begin(submatches), end(submatches))`.

-4- [...]

3130⁽⁰⁾. §[input.output] needs many addressof

Section: 30 [input.output] Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2018-06-30 Last modified: 2018-07-20

Priority: 0

View other [active issues](#) in [input.output].

View all other [issues](#) in [input.output].

View all issues with [Tentatively Ready](#) status.

Discussion:

There are 27 instances of `&sb` and one instance of `&rhs` in Clause 30 [input.output], each of which needs to use `addressof` because the operand has a user-provided template type parameter as an associated class and so the use of unary `&` is subject to ADL hijacking.

[2018-07-20 Status to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4750](#).

1. Change 30.5.5.3 [basic.ios.members] p16 as indicated:

```
basic_ios& copyfmt(const basic_ios& rhs);
```

-16- *Effects*: If (this == `&addressof(rhs)`) does nothing. [...]

2. Change 30.8.3.1 [istreamstream.cons] as indicated:

```
explicit basic_istreamstream(ios_base::openmode which);
```

-1- *Effects*: Constructs an object of class `basic_istreamstream<charT, traits>`, initializing the base class with `basic_istream<charT, traits>(&addressof(sb))` (30.7.4.1 [istream]) and initializing `sb` with `basic_stringbuf<charT, traits, Allocator>(which | ios_base::in)` (30.8.2.1 [stringbuf.cons]).

```
explicit basic_istreamstream(  
    const basic_string<charT, traits, Allocator>& str,  
    ios_base::openmode which = ios_base::in);
```

-2- *Effects*: Constructs an object of class `basic_istreamstream<charT, traits>`, initializing the base class with `basic_istream<charT, traits>(&addressof(sb))` (30.7.4.1 [istream]) and initializing `sb` with `basic_stringbuf<charT, traits, Allocator>(str, which | ios_base::in)` (30.8.2.1 [stringbuf.cons]).

```
basic_istreamstream(basic_istreamstream&& rhs);
```

-3- *Effects*: Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_stringbuf`. Next `basic_istream<charT, traits>::set_rdbuf(&addressof(sb))` is called to install the contained `basic_stringbuf`.

3. Change 30.8.3.3 [istreamstream.members] p1 as indicated:

```
basic_stringbuf<charT, traits, Allocator>* rdbuf() const;
```

-1- *Returns*: `const_cast<basic_stringbuf<charT, traits, Allocator>*>(&addressof(sb))`.

4. Change 30.8.4.1 [ostreamstream.cons] as indicated:

```
explicit basic_ostreamstream(ios_base::openmode which);
```

-1- *Effects*: Constructs an object of class `basic_ostreamstream<charT, traits>`, initializing the base class with `basic_ostream<charT, traits>(&addressof(sb))` (30.7.5.1 [ostream]) and initializing `sb` with `basic_stringbuf<charT, traits, Allocator>(which | ios_base::out)` (30.8.2.1 [stringbuf.cons]).

```
explicit basic_ostreamstream(  
    const basic_string<charT, traits, Allocator>& str,  
    ios_base::openmode which = ios_base::out);
```

-2- *Effects*: Constructs an object of class `basic_ostreamstream<charT, traits>`, initializing the base class with `basic_ostream<charT, traits>(&addressof(sb))` (30.7.5.1 [ostream]) and initializing `sb` with `basic_stringbuf<charT, traits, Allocator>(str, which | ios_base::out)` (30.8.2.1 [stringbuf.cons]).

```
basic_ostreamstream(basic_ostreamstream&& rhs);
```

-3- *Effects*: Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_stringbuf`. Next `basic_ostream<charT, traits>::set_rdbuf(&addressof(sb))` is called to install the contained `basic_stringbuf`.

5. Change 30.8.4.3 [ostreamstream.members] p1 as indicated:

```
basic_stringbuf<charT, traits, Allocator>* rdbuf() const;
```

-1- *Returns*: `const_cast<basic_stringbuf<charT, traits, Allocator>*>(&addressof(sb))`.

6. Change 30.8.5.1 [stringstream.cons] as indicated:

```
explicit basic_stringstream(ios_base::openmode which);
```

-1- *Effects:* Constructs an object of class `basic_stringstream<charT, traits>`, initializing the base class with `basic_istream<charT, traits>(&addressof(sb))` (30.7.4.6.1 [\[istream.cons\]](#)) and initializing sb with `basic_stringbuf<charT, traits, Allocator>(which)`.

```
explicit basic_stringstream(
    const basic_string<charT, traits, Allocator>& str,
    ios_base::openmode which = ios_base::out | ios_base::in);
```

-2- *Effects:* Constructs an object of class `basic_stringstream<charT, traits>`, initializing the base class with `basic_istream<charT, traits>(&addressof(sb))` (30.7.4.6.1 [\[istream.cons\]](#)) and initializing sb with `basic_stringbuf<charT, traits, Allocator>(str, which)`.

```
basic_stringstream(basic_stringstream&& rhs);
```

-3- *Effects:* Move constructs from the rvalue rhs. This is accomplished by move constructing the base class, and the contained `basic_stringbuf`. Next `basic_istream<charT, traits>::set_rdbuf(&addressof(sb))` is called to install the contained `basic_stringbuf`.

7. Change 30.8.5.3 [\[stringstream.members\]](#) p1 as indicated:

```
basic_stringbuf<charT, traits, Allocator>* rdbuf() const;
```

-1- *Returns:* `const_cast<basic_stringbuf<charT, traits, Allocator>*>(&addressof(sb))`.

8. Change 30.9.3.1 [\[ifstream.cons\]](#) as indicated:

```
basic_ifstream();
```

-1- *Effects:* Constructs an object of class `basic_ifstream<charT, traits>`, initializing the base class with `basic_istream<charT, traits>(&addressof(sb))` (30.7.4.1.1 [\[istream.cons\]](#)) and initializing sb with `basic_filebuf<charT, traits>()` (30.9.2.1 [\[filebuf.cons\]](#)).

```
explicit basic_ifstream(const char* s,
    ios_base::openmode mode = ios_base::in);
explicit basic_ifstream(const filesystem::path::value_type* s,
    ios_base::openmode mode = ios_base::in); // wide systems only; see 30.9.1 \[fstream.syn\]
```

-2- *Effects:* Constructs an object of class `basic_ifstream<charT, traits>`, initializing the base class with `basic_istream<charT, traits>(&addressof(sb))` (30.7.4.1.1 [\[istream.cons\]](#)) and initializing sb with `basic_filebuf<charT, traits>()` (30.9.2.1 [\[filebuf.cons\]](#)), then calls `rdbuf()->open(s, mode | ios_base::in)`. If that function returns a null pointer, calls `setstate(failbit)`.

[...]

```
basic_ifstream(basic_ifstream&& rhs);
```

-4- *Effects:* Move constructs from the rvalue rhs. This is accomplished by move constructing the base class, and the contained `basic_filebuf`. Next `basic_istream<charT, traits>::set_rdbuf(&addressof(sb))` is called to install the contained `basic_filebuf`.

9. Change 30.9.3.3 [\[ifstream.members\]](#) p1 as indicated:

```
basic_filebuf<charT, traits>* rdbuf() const;
```

-1- *Returns:* `const_cast<basic_filebuf<charT, traits>*>(&addressof(sb))`.

10. Change 30.9.4.1 [\[ofstream.cons\]](#) as indicated:

```
basic_ofstream();
```

-1- *Effects:* Constructs an object of class `basic_ofstream<charT, traits>`, initializing the base class with `basic_ostream<charT, traits>(&addressof(sb))` (30.7.5.1.1 [\[ostream.cons\]](#)) and initializing sb with `basic_filebuf<charT, traits>()` (30.9.2.1 [\[filebuf.cons\]](#)).

```
explicit basic_ofstream(const char* s,
    ios_base::openmode mode = ios_base::out);
explicit basic_ofstream(const filesystem::path::value_type* s,
    ios_base::openmode mode = ios_base::out); // wide systems only; see 30.9.1 \[fstream.syn\]
```

-2- *Effects:* Constructs an object of class `basic_ofstream<charT, traits>`, initializing the base class with `basic_ostream<charT, traits>(&addressof(sb))` (30.7.5.1.1 [\[ostream.cons\]](#)) and initializing sb with `basic_filebuf<charT, traits>()` (30.9.2.1 [\[filebuf.cons\]](#)), then calls `rdbuf()->open(s, mode | ios_base::out)`. If that function returns a null pointer, calls `setstate(failbit)`.

[...]

```
basic_ofstream(basic_ofstream&& rhs);
```

-4- *Effects:* Move constructs from the rvalue rhs. This is accomplished by move constructing the base class, and the contained `basic_filebuf`. Next `basic_ostream<charT, traits>::set_rdbuf(&addressof(sb))` is called to install the contained `basic_filebuf`.

11. Change 30.9.4.3 [\[ofstream.members\]](#) p1 as indicated:

```
basic_filebuf<charT, traits>* rdbuf() const;
```

-1- *Returns:* `const_cast<basic_filebuf<charT, traits>*>(&addressof(sb))`.

12. Change 30.9.5.1 [\[fstream.cons\]](#) as indicated:

```
basic_fstream();
```

-1- *Effects:* Constructs an object of class `basic_fstream<charT, traits>`, initializing the base class with `basic_istream<charT, traits>(&addressof(sb))` (30.7.4.6.1 [\[iostream.cons\]](#)) and initializing sb with `basic_filebuf<charT, traits>()`.

```
explicit basic_fstream(
    const char* s,
    ios_base::openmode mode = ios_base::in | ios_base::out);
explicit basic_fstream(
    const filesystem::path::value_type* s,
    ios_base::openmode mode = ios_base::in | ios_base::out); // wide systems only; see 30.9.1 \[fstream.syn\].
```

-2- *Effects:* Constructs an object of class `basic_fstream<charT, traits>`, initializing the base class with `basic_istream<charT, traits>(&addressof(sb))` (30.7.4.6.1 [\[iostream.cons\]](#)) and initializing sb with `basic_filebuf<charT, traits>()`, then calls `rdbuf()->open(s, mode)`. If that function returns a null pointer, calls `setstate(failbit)`.

[...]

```
basic_fstream(basic_fstream&& rhs);
```

-4- *Effects:* Move constructs from the rvalue rhs. This is accomplished by move constructing the base class, and the contained `basic_filebuf`. Next `basic_istream<charT, traits>::set_rdbuf(&addressof(sb))` is called to install the contained `basic_filebuf`.

13. Change 30.9.5.3 [\[fstream.members\]](#) p1 as indicated:

```
basic_filebuf<charT, traits>* rdbuf() const;
```

-1- *Returns:* `const_cast<basic_filebuf<charT, traits>*>(&addressof(sb))`.

14. Change 30.10.3.1 [\[syncstream.osyncstream.overview\]](#), class template `basic_osyncstream` synopsis, as indicated:

[Drafting note: The text shown below assumes the application of the proposed resolution of issue [3127](#).]

```
namespace std {
    template<class charT, class traits, class Allocator>
    class basic_osyncstream : public basic_ostream<charT, traits> {
    public:
        [...]

        // 30.10.3.4 \[syncstream.osyncstream.members\], member functions
        void emit();
        streambuf_type* get_wrapped() const noexcept;
        syncbuf_type* rdbuf() const noexcept { return const_cast<syncbuf_type*>(&addressof(sb)); }
        [...]
    };
}
```

15. Change 30.10.3.2 [\[syncstream.osyncstream.cons\]](#) p1 and p4 as indicated:

```
basic_osyncstream(streambuf_type* buf, const Allocator& allocator);
```

-1- *Effects:* Initializes sb from buf and allocator. Initializes the base class with `basic_ostream<charT, traits>(&addressof(sb))`.

-2- [...]

-3- [...]

```
basic_osyncstream(basic_osyncstream&& other) noexcept;
```

-4- *Effects:* Move constructs the base class and sb from the corresponding subobjects of other, and calls `basic_ostream<charT, traits>::set_rdbuf(&addressof(sb))`.

-5- [...]

[3131](#)⁽⁶⁾. `addressof` all the things

Section: 23.17.12 [\[time.parse\]](#), 24.3.2.7.1 [\[string.accessors\]](#), 24.4.2 [\[string.view.template\]](#), 26.2.1 [\[container.requirements.general\]](#), 27.2.4 [\[output.iterators\]](#), 27.2.6 [\[bidirectional.iterators\]](#), 31.7 [\[re.traits\]](#), 31.12.1 [\[re.register\]](#), 33.4.4.1 [\[thread.lock.guard\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2018-06-30 **Last modified:** 2018-07-20

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

Some additional instances where the library specification applies unary operator & when it should use `addressof`.

[2018-07-20 Status to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4750](#).

[Drafting note: Two uses of & in 27.5.1 [\[reverse.iterators\]](#) are not included in the wording below because the entire sentence is slated to be removed by a revision of [P0896](#), the One Ranges Proposal.]

1. Change 23.17.12 [\[time.parse\]](#) p4-5 as indicated:

```
template<class charT, class traits, class Alloc, class Parsable>
    unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev);
```

-4- *Remarks:* This function shall not participate in overload resolution unless

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp, &addressof(abbrev))
```

is a valid expression.

-5- *Returns:* A manipulator that, when extracted from a `basic_istream<charT, traits>` is, calls `from_stream(is, fmt.c_str(), tp, &addressof(abbrev))`.

2. Change 23.17.12 [\[time.parse\]](#) p8-9 as indicated:

```
template<class charT, class traits, class Alloc, class Parsable>
    unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev, minutes& offset);
```

-8- *Remarks:* This function shall not participate in overload resolution unless

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp, &addressof(abbrev), &offset)
```

is a valid expression.

-9- *Returns:* A manipulator that, when extracted from a `basic_istream<charT, traits>` is, calls `from_stream(is, fmt.c_str(), tp, &addressof(abbrev), &offset)`.

3. Change 24.3.2.7.1 [\[string.accessors\]](#) p1 and p4 as indicated:

```
const charT* c_str() const noexcept;
const charT* data() const noexcept;
```

-1- *Returns:* A pointer `p` such that `p + i == &addressof(operator[](i))` for each `i` in `[0, size())`.

-2- *Complexity:* Constant time.

-3- *Requires:* The program shall not alter any of the values stored in the character array.

```
charT* data() noexcept;
```

-4- *Returns:* A pointer `p` such that `p + i == &addressof(operator[](i))` for each `i` in `[0, size())`.

-5- *Complexity:* Constant time.

-6- *Requires:* The program shall not alter the value stored at `p + size()`.

4. Change 24.4.2.2 [\[string.view.iterators\]](#) p4 as indicated:

```
constexpr const_iterator begin() const noexcept;
constexpr const_iterator cbegin() const noexcept;
```

-4- *Returns:* An iterator such that

(4.1) — if `!empty()`, `&addressof(*begin()) == data_`,

(4.2) — otherwise, an unspecified value such that `[begin(), end())` is a valid range.

5. Change 24.4.2.6 [\[string.view.ops\]](#) p21 and p24 as indicated:

```
constexpr bool starts_with(charT x) const noexcept;
```

-21- *Effects:* Equivalent to: `return starts_with(basic_string_view(&addressof(x), 1));`

[...]

```
constexpr bool ends_with(charT x) const noexcept;
```

-24- *Effects:* Equivalent to: `return ends_with(basic_string_view(&addressof(x), 1));`

6. Change 24.4.2.7 [\[string.view.find\]](#) p5 as indicated:

-5- Each member function of the form

```
constexpr return-type F(charT c, size_type pos);
```

is equivalent to `return F(basic_string_view(&addressof(c), 1), pos);`

7. Edit 26.2.1 [\[container.requirements.general\]](#), Table 77 — "Container requirements", as indicated:

Table 77 — Container requirements

Expression	Return type	Operational semantics	Assertion/note pre/post-condition	Complexity
[...]				
((&a) -> a) -x()	void		the destructor is applied to every element of a; any memory obtained is deallocated.	linear
[...]				

8. Edit 27.2.4 [\[output.iterators\]](#), Table 90 — "Output iterator requirements (in addition to Iterator)", as indicated:

Table 90 — Output iterator requirements (in addition to Iterator)

Expression	Return type	Operational semantics	Assertion/note pre/post-condition
[...]			
++r	X&		&addressof(r) == &addressof(++r) . [...]
[...]			

9. Edit 27.2.6 [\[bidirectional.iterators\]](#), Table 92 — "Bidirectional iterator requirements (in addition to forward iterator)", as indicated:

Table 92 — Bidirectional iterator requirements (in addition to forward iterator)

Expression	Return type	Operational semantics	Assertion/note pre/post-condition
[...]			
--r	X&		[...] &addressof(r) == &addressof(--r) . [...]

10. Change 31.7 [\[re.traits\]](#) p6 as indicated:

```
template<class ForwardIterator>
string_type transform(ForwardIterator first, ForwardIterator last) const;
```

-6- *Effects*: As if by:

```
string_type str(first, last);
return use_facet<collate<charT>>(
    getloc()).transform(&str.begindata(), &str.begindata() + str.length());
```

11. Change 31.12.1.1 [\[re.regiter.cnstr\]](#) p2 as indicated:

```
regex_iterator(BidirectionalIterator a, BidirectionalIterator b,
    const regex_type& re,
    regex_constants::match_flag_type m = regex_constants::match_default);
```

-2- *Effects*: Initializes begin and end to a and b, respectively, sets pregex to ~~&addressof(re)~~, sets flags to m, then calls `regex_search(begin, end, match, *pregex, flags)`. If this call returns false the constructor sets `*this` to the end-of-sequence iterator.

12. Change 31.12.1.3 [\[re.regiter.deref\]](#) p2 as indicated:

```
const value_type* operator->() const;
```

-2- *Returns*: ~~&addressof(match)~~.

13. Change 33.4.4.1 [\[thread.lock.guard\]](#) p2-7 as indicated:

```
explicit lock_guard(mutex_type& m);
```

-2- *Requires*: If `mutex_type` is not a recursive mutex, the calling thread does not own the mutex m.

-3- *Effects*: ~~As if by~~ ~~Initializes pm with m. Calls~~ m.lock().

~~-4- Postconditions: &pm == &m~~

```
lock_guard(mutex_type& m, adopt_lock_t);
```

-5- *Requires*: The calling thread owns the mutex m.

-6- ~~Postconditions: &pm == &m~~ ~~Effects: Initializes pm with m.~~

-7- *Throws*: Nothing.

3132⁽ⁱ⁾. Library needs to ban macros named expects or ensures

Section: 20.5.4.3.2 [\[macro.names\]](#) Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2018-06-30 Last modified: 2018-08-10

Priority: 0

View other [active issues](#) in [\[macro.names\]](#).

View all other [issues](#) in [\[macro.names\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

expects and ensures are not technically described as *attribute-tokens* when used in a *contract-attribute-specifier*, so the existing prohibition in 20.5.4.3.2 [\[macro.names\]](#) doesn't apply to them.

The remaining special identifiers used by the contract attributes are all already covered by existing wording: `assert` is also a library name so falls under p1, `default` is a keyword, and both `axiom` and `audit` were added to Table 4.

[2018-07-20 Status to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4762](#).

1. Change 20.5.4.3.2 [\[macro.names\]](#) p2 as indicated:

-2- A translation unit shall not `#define` or `#undef` names lexically identical to keywords, to the identifiers listed in Table 4, ~~or~~ to the *attribute-tokens* described in 10.6 [\[dcl.attr\]](#), ~~or to the identifiers expects or ensures~~.

[3134](#)⁽ⁱ⁾. [\[fund.ts.v3\]](#) LFTSv3 contains extraneous [\[meta\]](#) variable templates that should have been deleted by P09961

Section: 3.3.1 [\[fund.ts.v3::meta.type.synop\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Thomas Köppe **Opened:** 2018-07-02 **Last modified:** 2018-07-20

Priority: 0

View other [active issues](#) in [\[fund.ts.v3::meta.type.synop\]](#).

View all other [issues](#) in [\[fund.ts.v3::meta.type.synop\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: [fund.ts.v3](#)

The LFTSv3 prospective-working-paper [N4758](#) lists a large number of type trait variable templates in [\[meta.type.synop\]](#) that are duplicates of corresponding ones in C++17. The paper [P0996R1](#) that was meant to rebase the LFTS on C++17 appears to have missed them.

[2018-07-20 Status to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4758](#).

1. Delete from 3.3.1 [\[fund.ts.v3::meta.type.synop\]](#) all variable templates starting at `is_void_v` up to and including `is_convertible_v` as indicated:

```
#include <type_traits>

namespace std::experimental {
    inline namespace fundamentals_v3 {

        // See C++17 §23.15.4.1, primary type categories
        template <class T> constexpr bool is_void_v
        = is_void<T>::value;

        [...]

        template <class From, class To> constexpr bool is_convertible_v
        = is_convertible<From, To>::value;

        // 3.3.2, Other type transformations
        template <class> class invocation_type; // not defined
        [...]
    } // inline namespace fundamentals_v3
} // namespace std::experimental
```

[3137](#)⁽ⁱ⁾. Header for `__cpp_lib_to_chars`

Section: 21.3.1 [\[support.limits.general\]](#) **Status:** [Tentatively Ready](#) **Submitter:** S. B.Tam **Opened:** 2018-07-03 **Last modified:** 2018-07-20

Priority: 0

View other [active issues](#) in [\[support.limits.general\]](#).

View all other [issues](#) in [\[support.limits.general\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

After acceptance of [P0941R2](#) into the working draft, in [\[support.limits.general\]](#), `__cpp_lib_to_chars` is required to be in `<utility>`. Since the relevant feature (`std::to_chars` and `std::from_chars`) is now in the header `<charconv>`, should the macro be defined in `<charconv>` instead of `<utility>`?

[Marshall provides P/R and context]

`std::to_chars`, etc were originally proposed for the header `<utility>` and SD-6 reflected that. Somewhere along the way, they was put into their own header `<charconv>`, but the document was never updated to reflect that.

When these macros were added to the standard, the (now incorrect) value was copied as well.

[2018-07-20 Status to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to (the Rapperswil post-mailing standard).

Change 21.3.1 [\[support.limits.general\]](#) (Table 35) as indicated:

Macro name	Value	Header(s)
<code>cpp_lib_to_chars</code>	201611L	<code><charconv></code> <code><utility></code>

3140⁽ⁱ⁾. *COMMON_REF* is unimplementable as specified

Section: 23.15.7.6 [\[meta.trans.other\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2018-07-07 **Last modified:** 2018-10-07

Priority: 0

View other [active issues](#) in [\[meta.trans.other\]](#).

View all other [issues](#) in [\[meta.trans.other\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

23.15.7.6 [\[meta.trans.other\]](#)/3.2 states:

[Let] $XREF(A)$ denote a unary class template T such that $T<U>$ denotes the same type as U with the addition of A 's cv and reference qualifiers, for a non-reference cv-unqualified type U ,

which is nonsensical: a specialization of a *class* template cannot possibly be a cv-qualified type or reference type. $XREF(A)$ must be a unary alias template.

[2018-09-11; Status set to Tentatively Ready after five positive votes on the reflector]

Proposed resolution:

This wording is relative to [N4762](#).

Change 23.15.7.6 [\[meta.trans.other\]](#) as indicated:

-3- Let:

(3.1) — $CREF(A)$ be `add_lvalue_reference_t<const remove_reference_t<A>>`,

(3.2) — $XREF(A)$ denote a unary ~~class~~`alias` template T such that $T<U>$ denotes the same type as U with the addition of A 's cv and reference qualifiers, for a non-reference cv-unqualified type U ,

(3.3) — $COPYCV(FROM, TO)$ be an alias for type TO with the addition of $FROM$'s top-level cv-qualifiers. [Example: `COPYCV(const int, volatile short)` is an alias for `const volatile short`. —end example]

3145⁽ⁱ⁾. `file_clock` breaks ABI for C++17 implementations

Section: 23.17.7.5 [\[time.clock.file\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Billy Robert O'Neal III **Opened:** 2018-07-26 **Last modified:** 2018-08-27

Priority: 1

View all issues with [Tentatively Ready](#) status.

Discussion:

It was pointed out in one of Eric's changes to libcpp [here](#) that [P0355](#) adds `file_clock`, which is intended to be the clock used for `std::filesystem::file_time_type`'s clock.

Unfortunately, this is an ABI break for implementations that are already shipping C++17 filesystem that did not call their clock type `std::file_clock`. For example, MSVC++'s is called `std::filesystem::_File_time_clock`.

We can keep much the same interface of [P0355](#) by making `file_clock` a typedef for an unspecified type. This technically changes the associated namespaces for expressions using that clock for the sake of ADL, but I can't imagine a user who cares, as clocks aren't generally called in ADL-able expressions, durations and time_points are.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4750](#).

1. Change 23.17.2 [\[time.syn\]](#), header `<chrono>` synopsis, as indicated:

```
[...]
// 23.17.7.5 [time.clock.file], classType file_clock
class using file_clock = unspecified;
[...]
```

2. Change 23.17.7.5 [time.clock.file] as indicated:

23.17.7.5 ClassType file_clock [time.clock.file]

3. Change 23.17.7.5.1 [time.clock.file.overview], class file_clock synopsis, as indicated:

```
namespace std::chrono {
    using file_clock = see below;
    class file_clock {
    public:
        using rep = a signed arithmetic type;
        using period = ratio<unspecified, unspecified>;
        using duration = chrono::duration<rep, period>;
        using time_point = chrono::time_point<file_clock>;
        static constexpr bool is_steady = unspecified;
        static time_point now() noexcept;
        // Conversion functions, see below
    };
}
```

-1- The clock file_clock is an alias for a type meeting the TrivialClock requirements (23.17.3 [time.clock.req]), uses a signed arithmetic type for file_clock::rep, and is used to create the time_point system used for file_time_type (30.11 [filesystems]). Its epoch is unspecified. [Note: The type file_clock denotes may be in a different namespace than std::chrono, such as std::filesystem. — end note]

4. Change 23.17.7.5.2 [time.clock.file.members] as indicated:

```
static time_point now();
```

~~1- Returns: A file_clock::time_point indicating the current time.~~

-2- The class file_clock shall type denoted by file_clock provides precisely one of the following two sets of static member functions: [...]

[2018-07-30 Priority set to 1 after reflector discussion; wording updates based on several discussion contributions.]

Previous resolution [SUPERSEDED]:

This wording is relative to N4762.

1. Change 23.17.2 [time.syn], header <chrono> synopsis, as indicated:

```
[...]
// 23.17.7.5 [time.clock.file], classType file_clock
class using file_clock = see below;
[...]
```

2. Change 23.17.7.5 [time.clock.file] as indicated:

23.17.7.5 ClassType file_clock [time.clock.file]

3. Change 23.17.7.5.1 [time.clock.file.overview], class file_clock synopsis, as indicated:

```
namespace std::chrono {
    using file_clock = see below;
    class file_clock {
    public:
        using rep = a signed arithmetic type;
        using period = ratio<unspecified, unspecified>;
        using duration = chrono::duration<rep, period>;
        using time_point = chrono::time_point<file_clock>;
        static constexpr bool is_steady = unspecified;
        static time_point now() noexcept;
        // Conversion functions, see below
    };
}
```

-1- The clock file_clock is an alias for a type meeting the TrivialClock requirements (23.17.3 [time.clock.req]), which uses a signed arithmetic type for file_clock::rep, file_clock is used to create the time_point system used for file_time_type (30.11 [filesystems]). Its epoch is unspecified, and noexcept(file_clock::now()) is true. [Note: The type file_clock denotes may be in a different namespace than std::chrono, such as std::filesystem. — end note]

4. Change 23.17.7.5.2 [time.clock.file.members] as indicated:

```
static time_point now();
```

~~1- Returns: A file_clock::time_point indicating the current time.~~

-2- The class `file_clock` type denoted by `file_clock` provides precisely one of the following two sets of static member functions: [...]

[2018-08-23 Batavia Issues processing: Minor wording changes, and status to "Tentatively Ready".]

Proposed resolution:

This wording is relative to [N4762](#).

1. Change 23.17.2 [\[time.syn\]](#), header <chrono> synopsis, as indicated:

```
[...]
// 23.17.7.5 \[time.clock.file\], classtype file_clock
classusing file_clock = see below;
[...]
```

2. Change 23.17.7.5 [\[time.clock.file\]](#) as indicated:

23.17.7.5 [Class Type](#) file_clock [\[time.clock.file\]](#)

3. Change 23.17.7.5.1 [\[time.clock.file.overview\]](#), class file_clock synopsis, as indicated:

```
namespace std::chrono {
    using file\_clock = see below;
    class file_clock {
    public:
        using rep = a signed arithmetic type;
        using period = ratio<unspecified, unspecified>;
        using duration = chrono::duration<rep, period>;
        using time_point = chrono::time_point<file_clock>;
        static constexpr bool is_steady = unspecified;
        static time_point now() noexcept;
        // Conversion functions, see below
    };
}
```

-1- ~~The clock~~ file_clock [is an alias for a type meeting the Cpp17TrivialClock requirements \(23.17.3 \[\\[time.clock.req\\]\]\(#\)\), and using a signed arithmetic type for file_clock::rep. file_clock is used to create the time_point system used for file_time_type \(30.11 \[\\[filesystems\\]\]\(#\)\). Its epoch is unspecified, and noexcept\(file_clock::now\(\)\) is true. \[Note: The type that file_clock denotes may be in a different namespace than std::chrono, such as std::filesystem. — end note\]](#)

4. Change 23.17.7.5.2 [\[time.clock.file.members\]](#) as indicated:

```
static time_point now();
```

~~-1- Returns: A file_clock::time_point indicating the current time.~~

-2- The type `file_clock` type denoted by `file_clock` provides precisely one of the following two sets of static member functions: [...]

3147⁽ⁱ⁾. Definitions of "likely" and "unlikely" are likely to cause problems

Section: 20.5.4.3.2 [\[macro.names\]](#) Status: [Tentatively Ready](#) Submitter: Casey Carter Opened: 2018-08-01 Last modified: 2018-08-27

Priority: 0

View other [active issues](#) in [\[macro.names\]](#).

View all other [issues](#) in [\[macro.names\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

20.5.4.3.2 [\[macro.names\]](#)/2 forbids a translation unit to define names "lexically identical to [...] the *attribute-tokens* described in 10.6 [\[dcl.attr\]](#)." We recently added the attribute-tokens likely and unlikely (10.6.7 [\[dcl.attr.likelihood\]](#)). These names are in extremely wide use as function-like macros in the open source community, forbidding users to define them breaks large amounts of code. (Reportedly Chromium contains 19 definitions each of "likely" and "unlikely" as function-like macros.)

Indeed, this issue came up during EWG discussion of [P0479R1](#) "Attributes for Likely and Unlikely Statements" in Kona, and EWG decided to keep the names "likely" and "unlikely" for the attribute tokens since the usage wouldn't conflict with defining them as function-like macros. 20.5.4.3.2 [\[macro.names\]](#)/2 should not break large amounts of existing code that doesn't actually conflict with the use of the `[[likely]]` and `[[unlikely]]` attributes.

[2018-08-20 Status to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4762](#).

1. Change 20.5.4.3.2 [\[macro.names\]](#) as indicated:

[Drafting Note: If both this proposed resolution and the proposed resolution of LWG [3132](#) are accepted, the text inserted by LWG [3132](#) should

precede the text added here.]

-2- A translation unit shall not `#define` or `#undef` names lexically identical to keywords, to the identifiers listed in Table 4, or to the *attribute-tokens* described in 10.6 [dcl.attr], **except that the names `likely` and `unlikely` may be defined as function-like macros (19.3 [cpp.replace]).**

3148⁽ⁱ⁾. `<concepts>` should be freestanding

Section: 20.5.1.3 [compliance] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2018-08-09 **Last modified:** 2018-08-27

Priority: 0

View all other [issues](#) in [compliance].

View all issues with [Tentatively Ready](#) status.

Discussion:

The design intent of the `<concepts>` header is that it contains only fundamental concept definitions and implementations of customization points that are used by those concept definitions. There should never be components in the header that require operating system support. Consequently, freestanding implementations can and should provide it. It is an oversight on the part of LWG - and in particular the author of [P0898R3 "Standard Library Concepts"](#) - that the `<concepts>` header is not required to be provided by freestanding implementations.

[2018-08 Batavia Monday issue prioritization]

Priority set to 0, status to 'Tentatively Ready'

Proposed resolution:

This wording is relative to [N4762](#).

1. In 20.5.1.3 [compliance], add a new row to [Table 21](#):

Table 21 — C++ headers for freestanding implementations		
Subclause		Header(s)
[...]	[...]	[...]
21.12 [support.runtime]	Other runtime support	<code><csdarg></code>
[concepts]	Concepts library	<code><concepts></code>
23.15 [meta]	Type traits	<code><type_traits></code>
[...]	[...]	[...]

3153⁽ⁱ⁾. `common` and `common_type` have too little in common

Section: 99 [concept.common] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2018-08-10 **Last modified:** 2018-08-27

Priority: 0

View other [active issues](#) in [concept.common].

View all other [issues](#) in [concept.common].

View all issues with [Tentatively Ready](#) status.

Discussion:

The `Common` concept when applied to types `T` and `U` requires that `T` and `U` are each `ConvertibleTo` (99 [concept.convertibleto]) their common type `common_type_t<T, U>`. `ConvertibleTo` requires both implicit and explicit conversions with equivalent results. The requirement for implicit conversion is notably not a requirement for specializing `common_type` as detailed in 23.15.7.6 [meta.trans.other]:

-5- Such a specialization need not have a member named `type`, but if it does, that member shall be a *typedef-name* for an accessible and unambiguous cv-qualified non-reference type `C` to which each of the types `T1` and `T2` is explicitly convertible.

which only requires *explicit* conversion to be valid. While it's not inconsistent that the `Common` concept's requirements are a refinement of the requirements for `common_type`, there's no good reason for this additional requirement. The stated design intent is to enable writing monomorphic predicates that can compare `Ts` with `Us` (and vice versa) by accepting two arguments of type `common_type_t<T, U>`, but this role has been superseded by the addition of `CommonReference` and `common_reference_t` to the ranges design. The existence of pairs of types that are only explicitly convertible to their common type suggests that using `Common` in this way would never be a fully generic solution in any case.

The only existing use of the `Common` concept in either the working draft or the Ranges proposal is as a soundness check on the [comparison](#) and [difference](#) operators of `counted_iterator`, none of which actually convert any argument to the common type in their normal operation. It would seem that we could strike the additional requirement without impacting the Ranges design, which would allow for future uses of the `Common` concept with types like `chrono::duration` (23.17.5 [time.duration]) which sometimes provide only explicit conversion to a common type.

Notably, removing the requirement for implicit conversion will also make the `Common` concept consistent with the description in 99 [concept.common] p1: "If `T` and `U` can both be explicitly converted to some third type, `C`, then `T` and `U` share a common type, `C`."

[2018-08 Batavia Monday issue prioritization]

P0; Status to 'Tentatively Ready' after adding two semicolons to the P/R.

Proposed resolution:

This wording is relative to [N4762](#).

- Modify the definition of `Common` in 99 [concept.common] as follows:

```
template<class T, class U>
concept Common =
    Same<common_type_t<T, U>, common_type_t<U, T>> &&
    ConvertibleTo<T, common_type_t<T, U>> &&
    ConvertibleTo<U, common_type_t<T, U>> &&
    requires {
        static_cast<common_type_t<T, U>>(declval<T>());
        static_cast<common_type_t<T, U>>(declval<U>());
    } &&
    CommonReference<
        add_lvalue_reference_t<const T>,
        add_lvalue_reference_t<const U>> &&
    CommonReference<
        add_lvalue_reference_t<common_type_t<T, U>>,
        common_reference_t<
            add_lvalue_reference_t<const T>,
            add_lvalue_reference_t<const U>>>;
```

[3154^{\(1\)}](#). `Common` and `CommonReference` have a common defect

Section: 99 [concept.common] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2018-08-10 **Last modified:** 2018-08-27

Priority: 0

View other [active issues](#) in [concept.common].

View all other [issues](#) in [concept.common].

View all issues with [Tentatively Ready](#) status.

Discussion:

The semantic requirements of both `Common` (99 [concept.common]):

-2- Let `C` be `common_type_t<T, U>`. Let `t` be a function whose return type is `T`, and let `u` be a function whose return type is `U`. `Common<T, U>` is satisfied only if:

(2.1) — `C(t())` equals `C(u())` if and only if `t()` is an equality-preserving expression ([concepts.equality]).

(2.2) — `C(u())` equals `C(u())` if and only if `u()` is an equality-preserving expression ([concepts.equality]).

and similarly `CommonReference` ([concept.commonreference]):

-2- Let `C` be `common_reference_t<T, U>`. Let `t` be a function whose return type is `T`, and let `u` be a function whose return type is `U`. `CommonReference<T, U>` is satisfied only if:

(2.1) — `C(t())` equals `C(u())` if and only if `t()` is an equality-preserving expression ([concepts.equality]).

(2.2) — `C(u())` equals `C(u())` if and only if `u()` is an equality-preserving expression.

don't properly reflect the intended design that conversions to the common type / common reference type are identity-preserving: in other words, that converting two values to the common type produces equal results if and only if the values were initially equal. The phrasing "`C(E)` equals `C(E)` if and only if `E` is an equality-preserving expression" is also clearly defective regardless of the intended design: the assertion "`E` is not equality-preserving" does not imply that every evaluation of `E` produces different results.

[2018-08 Batavia Monday issue prioritization]

Priority set to 0, status to 'Tentatively Ready'

Proposed resolution:

This wording is relative to [N4762](#).

- Modify [concept.commonref] p2 as follows:

-2- Let `C` be `common_reference_t<T, U>`. Let `t` be a function whose return type is `T` and `t1` and `t2` be equality-preserving expressions ([concepts.equality]) such that `decltype((t1))` and `decltype((t2))` are each `T`, and let `u` be a function whose return type is `U` and `u1` and `u2` be equality-preserving expressions such that `decltype((u1))` and `decltype((u2))` are each `U`. `T` and `U` model `CommonReference<T, U>` is satisfied only if:

(2.1) — `C(t1)` equals `C(t2)` if and only if `t1` equals `t2` and is an equality-preserving expression ([concepts.equality]).

(2.2) — `C(u1)` equals `C(u2)` if and only if `u1` equals `u2` is an equality-preserving expression.

- Modify 99 [concept.common] p2 similarly:

-2- Let `C` be `common_type_t<T, U>`. Let `t` be a function whose return type is `T` and `t1` and `t2` be equality-preserving expressions ([concepts.equality]) such that `decltype((t1))` and `decltype((t2))` are each `T`, and let `u` be a function whose return type is `U` and `u1` and `u2` be equality-preserving expressions such that `decltype((u1))` and `decltype((u2))` are each `U`. `T` and `U` model `Common<T, U>` is satisfied only if:

(2.1) — `C(t1)` equals `C(t2)` if and only if `t1` equals `t2` and is an equality-preserving expression ([concepts.equality]).

(2.2) — `C(u1)` equals `C(u2)` if and only if `u1` equals `u2` is an equality-preserving expression ([concepts.equality]).

3160⁽¹⁾. `atomic_ref()` = delete; should be deleted

Section: 32.6 [\[atomics.ref.generic\]](#) Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2018-10-01 Last modified: 2018-10-07

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

`atomic_ref` has a deleted default constructor, which causes pointless ambiguities in cases like:

```
void meow(atomic_ref<int>);  
void meow(some_default_constructible_struct);  
  
meow({});
```

It should have no default constructor rather than a deleted one. (Note that it has other user-defined constructors and so cannot be an aggregate under any definition.)

[2018-10-06 Status to Tentatively Ready after seven positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4762](#).

- Edit 32.6 [\[atomics.ref.generic\]](#), class template `atomic_ref` synopsis, as indicated:

```
[...]  
atomic_ref() = delete;  
atomic_ref& operator=(const atomic_ref&) = delete;  
[...]
```

- Edit 32.6.2 [\[atomics.ref.int\]](#), class template specialization `atomic_ref<integral>` synopsis, as indicated:

```
[...]  
atomic_ref() = delete;  
atomic_ref& operator=(const atomic_ref&) = delete;  
[...]
```

- Edit 32.6.3 [\[atomics.ref.float\]](#), class template specialization `atomic_ref<floating-point>` synopsis, as indicated:

```
[...]  
atomic_ref() = delete;  
atomic_ref& operator=(const atomic_ref&) = delete;  
[...]
```

- Edit 32.6.4 [\[atomics.ref.pointer\]](#), class template specialization `atomic_ref<T*>` synopsis, as indicated:

```
[...]  
atomic_ref() = delete;  
atomic_ref& operator=(const atomic_ref&) = delete;  
[...]
```